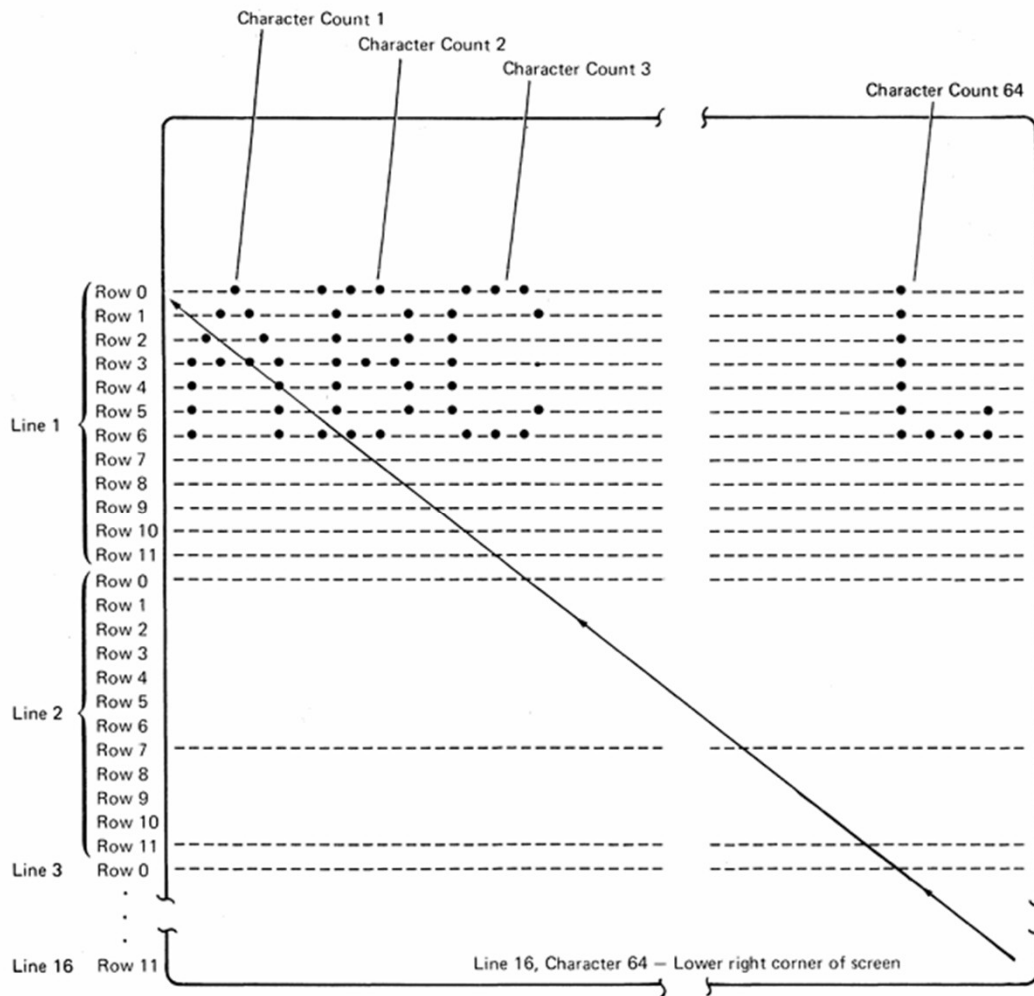


# Xiphod's Notes on IBM5100 Scroller PALM Code

- June 2025



6x7 pixel characters.

Columns 2 pixel padded, rows 5 pixel padded.

```
0010 PRINT 'HELLO'
0020 PRINT 'WORLD'
RUN
HELLO
WORLD

READY
```



```

LINE 1  0x0200
LINE 2  0x0240
LINE 3  0x0280
LINE 4  0x02C0
LINE 5  0x0300
LINE 6  0x0340
LINE 7  0x0380
LINE 8  0x03C0
LINE 9  0x0400
LINE 10 0x0440
LINE 11 0x0480
LINE 12 0x04C0
LINE 13 0x0500
LINE 14 0x0540
LINE 15 0x0580
LINE 16 0x05C0
0x0600

```

```

1 2 3 4 5 6
123456789012345678901234567890123456789012345678901234
LINE 1 0x0200
LINE 2 0x0240
LINE 3 0x0280
LINE 4 0x02C0
LINE 5 0x0300
LINE 6 0x0340
LINE 7 0x0380
LINE 8 0x03C0
LINE 9 0x0400
LINE 10 0x0440
LINE 11 0x0480
LINE 12 0x04C0
LINE 13 0x0500
LINE 14 0x0540
LINE 15 0x0580
LINE 16 0x05C0
0x0600

```

```

ROW 1 0x1000
ROW 2 ???
ROW 3 ???
ROW 4 ???
ROW 5 ???
ROW 6 ???
ROW 7 ???
ROW 8 ???
ROW 9 ???
ROW 10 ???

```

```

  AAAA  BBBB  CCC  DDDDD  EEEEE  FFFFF  GGGG  HH  HH  IIIII  JJ  KK  KK  LL
_AAAAA_ BBBB_ CCCCC_ DDDDD_ EEEEE_ FFFFF_ GGGGG_ HH  HH  IIIII  JJ  KK  KK  LL
_AA  AA_ BB  BB_ CC  CC_ DD  DD_ EE  _FF  _GG  GG_ HH  HH_ II  _JJ  KK  KK  _LL
_AA  AA_ BB  BB_ CC  _DD  DD_ EE  _FF  _GG  _HH  HH_ II  _JJ  KK  KK  _LL
_AAAAA_ BBBB_ CC  _DD  DD_ EEEE  _FFFF  _GG  _HHHHH  II  _JJ  KKK  _LL
_AAAAA_ BB  BB_ CC  _DD  DD_ EEEE  _FFFF  _GG  GGG  HHHHH  II  _JJ  KKK  _LL
_AA  AA_ BB  BB_ CC  _DD  DD_ EE  _FF  _GG  GGG  HH  HH_ II  _JJ  JJ  KK  KK  _LL
_AA  AA_ BB  BB_ CC  CC_ DD  DD_ EE  _FF  _GG  GG  HH  HH_ II  _JJ  JJ  KK  KK  _LL
_AA  AA_ BBBB_ CCCCC_ DDDDD_ EEEEE  _FF  _GGGGG  HH  HH  IIIII  JJJJ  KK  KK  LLLLL
_AA  AA_ BBBB_ CCC  _DDDD_ EEEEE  _FF  _GGG  HH  HH  IIIII  JJJ  KK  KK  LLLLL

```

Define “big letters” using 8x10 bytes.  
Means 8 characters fills the width of the 64-column screen.



|         |        |        |                                                        |   |   |   |   |   |  |
|---------|--------|--------|--------------------------------------------------------|---|---|---|---|---|--|
|         |        |        | 1                                                      | 2 | 3 | 4 | 5 | 6 |  |
|         |        |        | 123456789012345678901234567890123456789012345678901234 |   |   |   |   |   |  |
| LINE 1  | 0x0200 |        |                                                        |   |   |   |   |   |  |
| LINE 2  | 0x0240 |        |                                                        |   |   |   |   |   |  |
| LINE 3  | 0x0280 |        |                                                        |   |   |   |   |   |  |
| LINE 4  | 0x02C0 | ROW 1  | 0x1000                                                 |   |   |   |   |   |  |
| LINE 5  | 0x0300 | ROW 2  | 0x1060                                                 |   |   |   |   |   |  |
| LINE 6  | 0x0340 | ROW 3  | 0x10C0                                                 |   |   |   |   |   |  |
| LINE 7  | 0x0380 | ROW 4  | 0x1120                                                 |   |   |   |   |   |  |
| LINE 8  | 0x03C0 | ROW 5  | 0x1180                                                 |   |   |   |   |   |  |
| LINE 9  | 0x0400 | ROW 6  | 0x11E0                                                 |   |   |   |   |   |  |
| LINE 10 | 0x0440 | ROW 7  | 0x1240                                                 |   |   |   |   |   |  |
| LINE 11 | 0x0480 | ROW 8  | 0x12A0                                                 |   |   |   |   |   |  |
| LINE 12 | 0x04C0 | ROW 9  | 0x1300                                                 |   |   |   |   |   |  |
| LINE 13 | 0x0500 | ROW 10 | 0x1360                                                 |   |   |   |   |   |  |
| LINE 14 | 0x0540 |        |                                                        |   |   |   |   |   |  |
| LINE 15 | 0x0580 |        |                                                        |   |   |   |   |   |  |
| LINE 16 | 0x05C0 |        |                                                        |   |   |   |   |   |  |
|         | 0x0600 |        |                                                        |   |   |   |   |   |  |

|        |        |        |       |        |        |       |      |        |        |        |        |       |      |
|--------|--------|--------|-------|--------|--------|-------|------|--------|--------|--------|--------|-------|------|
| AAAA   | BBBBB  | CCCC   | DDDDD | EEEEEE | FFFFFF | GGGG  | HH   | HH     | IIIIII | JJ     | KK     | KK    | LL   |
| AAAAAA | BBBBBB | CCCCC  | DDDDD | EEEEEE | FFFFFF | GGGGG | HH   | HH     | IIIIII | JJ     | KK     | KK    | LL   |
| AA     | AA     | BB     | BB    | CC     | CC     | DD    | DD   | EE     | FF     | GG     | GG     | HH    | HH   |
| AA     | AA     | BB     | BB    | CC     |        | DD    | DD   | EE     | FF     | GG     |        | HH    | HH   |
| AAAAAA | BBBBB  | CC     |       | DD     | DD     | EEEE  | FFFF | GG     |        | HHHHHH | II     | JJ    | KKKK |
| AAAAAA | BB     | BB     | CC    |        | DD     | DD    | EEEE | FFFF   | GG     | GGG    | HHHHHH | II    | JJ   |
| AA     | AA     | BB     | BB    | CC     |        | DD    | DD   | EE     | FF     | GG     | GGG    | HH    | HH   |
| AA     | AA     | BB     | BB    | CC     | CC     | DD    | DD   | EE     | FF     | GG     | GG     | HH    | HH   |
| AA     | AA     | BBBBBB | CCCCC | DDDDD  | EEEEEE | FF    |      | GGGGGG | HH     | HH     | IIIIII | JJJJJ | KK   |
| AA     | AA     | BBBBB  | CCCC  | DDDDD  | EEEEEE | FF    |      | GGGG   | HH     | HH     | IIIIII | JJJ   | KK   |

8 bytes per character x 12 characters = 96 characters per row  
 For each row, increment address by 96.  
 96 decimal == 0x60 (6\*16<sup>1</sup> + 0\*16<sup>0</sup> = 96)

```

LINE 1      0x0200
LINE 2      0x0240
LINE 3      0x0280
LINE 4      0x02C0
LINE 5      0x0300
LINE 6      0x0340
LINE 7      0x0380
LINE 8      0x03C0
LINE 9      0x0400
LINE 10     0x0440
LINE 11     0x0480
LINE 12     0x04C0
LINE 13     0x0500
LINE 14     0x0540
LINE 15     0x0580
LINE 16     0x05C0
0x0600

```

## WORKING\_ADDRESS

|        |        |
|--------|--------|
| ROW 1  | 0x1008 |
| ROW 2  | 0x1068 |
| ROW 3  | 0x10C8 |
| ROW 4  | 0x1128 |
| ROW 5  | 0x1188 |
| ROW 6  | 0x11E8 |
| ROW 7  | 0x1248 |
| ROW 8  | 0x12A8 |
| ROW 9  | 0x1308 |
| ROW 10 | 0x1368 |

1 2 3 4 5 6

|        |        |       |       |      |        |        |       |      |      |      |    |       |    |
|--------|--------|-------|-------|------|--------|--------|-------|------|------|------|----|-------|----|
| AAAA   | BBBB   | CCCC  | DDDD  | EEEE | FFFF   | GGG    | HH    | HH   | IIII | JJ   | KK | KK    | LL |
| AAAAAA | BBBBBB | CCCCC | DDDDD | EEEE | FFFFFF | GGGGG  | HH    | HH   | IIII | JJ   | KK | KK    | LL |
| AA AA  | BB BB  | CC CC | DD DD | EE   | FF     | GG GG  | HH HH | II   | JJ   | KK   | KK | LL    |    |
| AA AA  | BB BB  | CC    | DD DD | EE   | FF     | GG     | HH HH | II   | JJ   | KK   | KK | LL    |    |
| AAAAAA | BBBB   | CC    | DD DD | EEEE | FFFF   | GG     | HHHHH | II   | JJ   | KKKK | LL |       |    |
| AAAAAA | BB BB  | CC    | DD DD | EEEE | FFFF   | GG GGG | HHHHH | II   | JJ   | KKKK | LL |       |    |
| AA AA  | BB BB  | CC    | DD DD | EE   | FF     | GG GGG | HH HH | II   | JJ   | JJ   | KK | KK    | LL |
| AA AA  | BB BB  | CC CC | DD DD | EE   | FF     | GG GG  | HH HH | II   | JJ   | JJ   | KK | KK    | LL |
| AA AA  | BBBBBB | CCCCC | DDDDD | EEEE | FF     | GGGGG  | HH HH | IIII | JJJJ | KK   | KK | LLLLL |    |
| AA AA  | BBBB   | CCCC  | DDDD  | EEEE | FF     | GGGG   | HH HH | IIII | JJJ  | KK   | KK | LLLLL |    |





8 bytes per character x **120 characters** = 960 characters per row  
960 characters per row x 10 rows = 9600 characters (bytes) **(yikes!)**

Instead, we can represent each row of a “big letter” as a hex sequence.

|                    |    |      |      |    |    |                                                    |
|--------------------|----|------|------|----|----|----------------------------------------------------|
| ____ AAAA ____     | == | 0001 | 1110 | == | 1E | 80 bytes (8x10) now expressed<br>in only 10 bytes. |
| ____ AAAAAA ____   | == | 0011 | 1111 | == | 3F |                                                    |
| ____ AA    AA ____ | == | 0011 | 0011 | == | 33 | Add a symbol header, becomes<br>now 12 bytes.      |
| ____ AA    AA ____ | == | 0011 | 0011 | == | 33 |                                                    |
| ____ AAAAAA ____   | == | 0011 | 1111 | == | 3F |                                                    |
| ____ AAAAAA ____   | == | 0011 | 1111 | == | 3F |                                                    |
| ____ AA    AA ____ | == | 0011 | 0011 | == | 33 |                                                    |
| ____ AA    AA ____ | == | 0011 | 0011 | == | 33 |                                                    |
| ____ AA    AA ____ | == | 0011 | 0011 | == | 33 |                                                    |
| ____ AA    AA ____ | == | 0011 | 0011 | == | 33 |                                                    |

Then we can programmatically compute a big-letter sequence from a given “normal string” sequence.

8 bytes per character x **120 characters** = 960 characters per row  
960 characters per row x 10 rows = 9600 characters (bytes) **(yikes!)**

Instead, we can represent each row of a “big letter” as a hex sequence.

|                |    |      |      |    |    |   |       |   |   |   |   |   |
|----------------|----|------|------|----|----|---|-------|---|---|---|---|---|
| ____ AAAA ____ | == | 0001 | 1110 | == | 1E | → | — — — | X | X | X | X | — |
| ____ AAAAAA    | == | 0011 | 1111 | == | 3F | → | — —   | X | X | X | X | X |
| ____ AA AA     | == | 0011 | 0011 | == | 33 | → | — —   | X | X | — | — | X |
| ____ AA AA     | == | 0011 | 0011 | == | 33 | → | — —   | X | X | — | — | X |
| ____ AAAAAA    | == | 0011 | 1111 | == | 3F | → | — —   | X | X | X | X | X |
| ____ AAAAAA    | == | 0011 | 1111 | == | 3F | → | — —   | X | X | X | X | X |
| ____ AA AA     | == | 0011 | 0011 | == | 33 | → | — —   | X | X | — | — | X |
| ____ AA AA     | == | 0011 | 0011 | == | 33 | → | — —   | X | X | — | — | X |
| ____ AA AA     | == | 0011 | 0011 | == | 33 | → | — —   | X | X | — | — | X |
| ____ AA AA     | == | 0011 | 0011 | == | 33 | → | — —   | X | X | — | — | X |

NOTE: Would like to control what on-system display-symbol is used for any expanded “big letter.”



| U420 |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |   |   | =CONCAT(L420,M420,N420,O420,P420,Q420,R420,S420) |      |           |           |
|------|----|----|----|----|----|----|----|----|---|---|---|---|---|---|---|---|---|---|---|---|--------------------------------------------------|------|-----------|-----------|
|      | A  | B  | C  | D  | E  | F  | G  | H  | I | J | K | L | M | N | O | P | Q | R | S | T | U                                                | V    | W         | X         |
| 419  |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |   |   |                                                  | 38   | 5600      | db \$5600 |
| 420  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 11111111                                         | 456  | FF        | db \$FF   |
| 421  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 11111111                                         |      | FF        | db \$FF   |
| 422  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 11111111                                         |      | FF        | db \$FF   |
| 423  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 11111111                                         |      | FF        | db \$FF   |
| 424  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 11111111                                         |      | FF        | db \$FF   |
| 425  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |   | 11111110                                         |      | FE        | db \$FE   |
| 426  | XX | XX | XX | XX |    |    |    | XX |   |   |   | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 |   | 11110001                                         |      | F1        | db \$F1   |
| 427  |    |    |    |    | XX | XX | XX | XX |   |   |   | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |   | 00001111                                         |      | 0F        | db \$0F   |
| 428  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 429  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 430  |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |   |   | 39                                               | 5600 | db \$5600 |           |
| 431  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         | 468  | FF        | db \$FF   |
| 432  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 433  | XX | XX |    |    |    |    | XX | XX |   |   |   | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |   | 11000011                                         |      | C3        | db \$C3   |
| 434  | XX |    | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 0 | 1 | 1 | 1 | 1 | 0 | 1 |   | 10111101                                         |      | BD        | db \$BD   |
| 435  |    | XX | XX | XX | XX | XX | XX | XX |   |   |   | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |   | 01111110                                         |      | 7E        | db \$7E   |
| 436  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 437  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 438  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 439  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 440  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 441  |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |   |   | 40                                               | 5600 | db \$5600 |           |
| 442  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         | 480  | FF        | db \$FF   |
| 443  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 444  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 445  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 446  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 447  |    | XX | XX | XX | XX | XX | XX | XX |   |   |   | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 01111111                                         |      | 7F        | db \$7F   |
| 448  | XX |    |    |    | XX | XX | XX | XX |   |   |   | 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |   | 10001111                                         |      | 8F        | db \$8F   |
| 449  | XX | XX | XX | XX |    |    |    |    |   |   |   | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |   | 11110000                                         |      | F0        | db \$F0   |
| 450  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 451  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 452  |    |    |    |    |    |    |    |    |   |   |   |   |   |   |   |   |   |   |   |   | 41                                               | 5600 | db \$5600 |           |
| 453  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         | 492  | FF        | db \$FF   |
| 454  | XX | XX | XX | XX | XX | XX | XX | XX |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |   | 11111111                                         |      | FF        | db \$FF   |
| 455  | XX | XX | XX | XX | XX | XX |    |    |   |   |   | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 |   | 11111100                                         |      | FC        | db \$FC   |

|    |    |
|----|----|
| 56 | 00 |
|----|----|

“big letter” #38

IBM symbol index

Pad/future settings

“big letter” #39

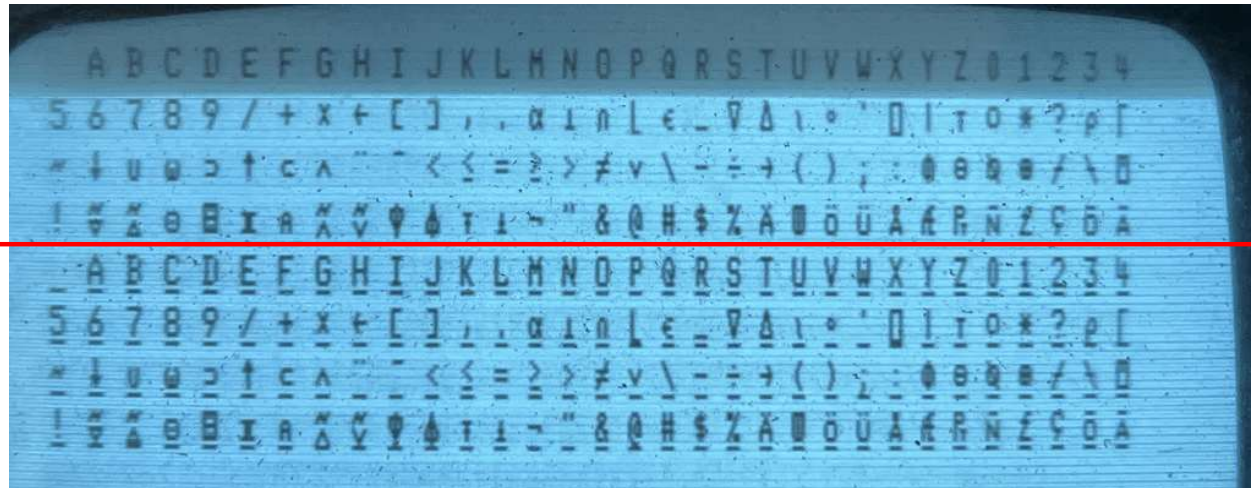
Symbol index offset can be greater than 255 (0xFF)

“big letter” #40



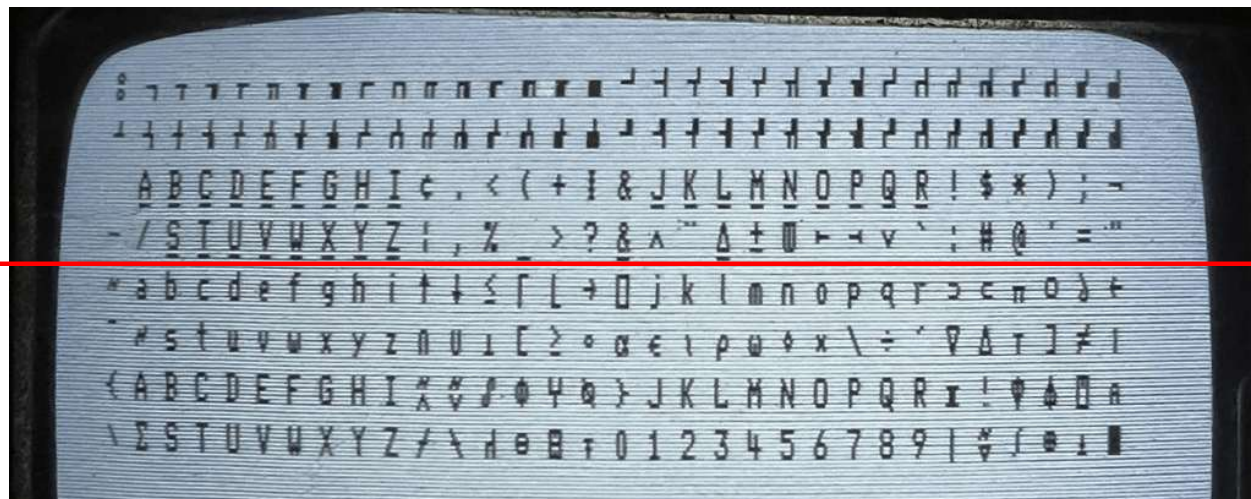
## IBM 5100

0 (00)  
32 (20)  
64 (40)  
96 (60)  
128 (80)  
160 (A0)  
192 (C0)  
224 (E0)



## IBM 5110

0 (00)  
32 (20)  
64 (40)  
96 (60)  
128 (80)  
160 (A0)  
192 (C0)  
224 (E0)



LINE 1 0x0200  
LINE 2 0x0240  
LINE 3 0x0280  
LINE 4 0x02C0  
LINE 5 0x0300  
LINE 6 0x0340  
LINE 7 0x0380  
LINE 8 0x03C0  
LINE 9 0x0400  
LINE 10 0x0440  
LINE 11 0x0480  
LINE 12 0x04C0  
LINE 13 0x0500  
LINE 14 0x0540  
LINE 15 0x0580  
LINE 16 0x05C0  
0x0600

WORKING\_ADDRESS

ROW 1 0x101E  
ROW 2 0x107E  
ROW 3 0x10DE  
ROW 4 0x113E  
ROW 5 0x119E  
ROW 6 0x11FE  
ROW 7 0x125E  
ROW 8 0x12BE  
ROW 9 0x131E  
ROW 10 0x137E

ROW 1 0x1000  
ROW 2 0x1060  
ROW 3 0x10C0  
ROW 4 0x1120  
ROW 5 0x1180  
ROW 6 0x11E0  
ROW 7 0x1240  
ROW 8 0x12A0  
ROW 9 0x1300  
ROW 10 0x1360

00020406  
080A0C0E  
10121416  
181A1C  
1E...

|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |
|  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |

# ASCII to “big letters” Translation

```
...
include      "cp5100.inc"          (cp == codepage)
codepage     cp5100                (activate the designated codepage)
...
db  "          VCF DALLAS 2025 - IBM 5100 SCROLLER          " ← ASCII "data byte" (db)
becomes encoded as a byte-sequence...
    $00, $00, $00, $00, $00, $00, $00, $00, $14, $03, $06, $00, $04, $01, ...
```

**But we will need 16-bit offsets... use "data words" (dw)**

```
dw  $01BC,$01BC,$01BC,$01BC,$01BC,$01BC,$01BC,$01BC          ; 8
dw  $00FC,$0018,$003C,$01BC                                   ; "VCF "          ; 4
dw  $0024,$0000,$0084,$0084,$0000,$00D8,$01BC                ; "DALLAS "          ; 7
dw  $0150,$01B0,$0150,$0174,$01BC,$0138,$01BC                ; "2025 - "          ; 7
dw  $0060,$000C,$0090,$01BC,$0174,$0144,$01B0,$01B0,$01BC    ; "IBM 5100 "          ; 9
dw  $00D8,$0018,$00CC,$00A8,$0084,$0084,$0030,$00CC,$01BC    ; "SCROLLER "          ; 9
dw  $01BC,$01BC,$01BC,$01BC,$01BC,$01BC,$01BC,$01BC          ; 8 +
dw  $FFFF                                                      ; --
                                          ; 52 len.
```

# Initial Look at PALM Assembly Code

```
ADDR_SCREEN          equ $0200    ; Start address of CRT display (top left)
BG_SYMBOL_W          equ $0000    ; used in Clear Screen function (_W for "word")
                                ; BG for "background" (00 = space/blank on 5100)
...
ClrScr:
    LWI    R14, ADDR_SCREEN        ; R14 used as the screen offset [ LWI = Load Word Immediate ]
    LWI    R9,  BG_SYMBOL_W        ; R9 full word is used as screen code that is
                                ; to be written (e.g. " ")
    LBI    R3,  $06                ; Lo R3 = $06 (will be used to check to see if our address
                                ; pointer has reached $06xx yet) [ LBI = Load Byte Immediate ]
cs_1:
    MOVE   (R14)+, R9              ; RWS[R14] = R9, then R14 = R14 + 2 (next address)
                                ; (this does the actual character display update)
    SBSH   R14, R3                 ; Have we reached $0600 yet? (one past end of screen)
                                ; [ Skip if All Set Bits of R3 are Also Set in Hi R14 ]
                                ; NOTE: The above SBSH will set R1 = R0+4 -- this
                                ; is in case the code after the jump goes to a
                                ; subroutine -- as documented in IBM 5100
                                ; MIM Appendix C-16)
    BRA    cs_1                   ; Did not match yet, repeat again for next address
    RET    R2                     ; We "skip" (jump) to here if all the R3 bits match (return to caller)
...
```



[illegible]

| HEX.        | ADDRESS    | DESCRIPTION                              |
|-------------|------------|------------------------------------------|
| 0000 - 001F | (R0) (R15) | LEVEL 0 REGISTERS<br>(NORMAL OPERATION)  |
| 0020 - 003F |            | LEVEL 1 REGISTERS<br>(COMM. / SERIAL IO) |
| 0040 - 005F |            | LEVEL 2 REGISTERS<br>(PRINTER / TAPE)    |
| 0060 - 007F |            | LEVEL 3 REGISTERS<br>(KEYBOARD)          |
| <hr/>       |            |                                          |
| 0080 - 01FF |            | RWS / RAM (SYSTEM)                       |
| 0200 - 05FF |            | DISPLAY MEMORY MAP<br>(64x16 = 1024)     |
| 0600 - 3FFF |            | RWS / RAM (16KB)                         |
| 4000 - 7FFF |            | RWS / RAM (32KB)                         |
| 8000 - FFFF |            | RWS / RAM (64KB)                         |

# P.A.L.M. INSTRUCTION SET

|    | A        | B                                  | C         | D                            | E         | F           | G           | H        | I | J                                  |
|----|----------|------------------------------------|-----------|------------------------------|-----------|-------------|-------------|----------|---|------------------------------------|
| 1  |          | From AS Alfred Arnold Assembler... |           |                              | Nibble1   | Nibble2     | Nibble3     | Nibble4  |   |                                    |
| 2  | Corti    | IBM                                | Chapter 2 |                              | OpCode    | Data or Reg | Mask or Reg | Modifier |   |                                    |
| 3  | NOP      |                                    |           | (Corti psuedo instruction)   | "0004"    |             |             |          |   |                                    |
| 4  | HALT     |                                    |           | (Corti psuedo instruction)   | "0000"    |             |             |          |   |                                    |
| 5  | DEC2     |                                    |           | (Corti psuedo instruction)   | Uses MVM2 |             |             |          |   |                                    |
| 6  | DEC      |                                    |           | (Corti psuedo instruction)   | Uses MVM1 |             |             |          |   |                                    |
| 7  | INC      |                                    |           | (Corti psuedo instruction)   | Uses MVP1 |             |             |          |   |                                    |
| 8  | INC2     |                                    |           | (Corti psuedo instruction)   | Uses MVP2 |             |             |          |   |                                    |
| 9  | MOVB     |                                    |           | (Corti psuedo instruction)   |           |             |             |          |   |                                    |
| 10 | LWI      |                                    |           | (Corti psuedo instruction)   |           |             |             |          |   |                                    |
| 11 | BRA      |                                    |           | (Corti psuedo instruction)   |           |             |             |          |   |                                    |
| 12 | JMP      |                                    |           | (Corti psuedo instruction)   |           |             |             |          |   |                                    |
| 13 | CALL     |                                    |           | (Corti psuedo instruction)   |           |             |             |          |   |                                    |
| 14 | RCALL    |                                    |           | (Corti psuedo instruction)   |           |             |             |          |   |                                    |
| 15 | RET      |                                    |           | (Corti psuedo instruction)   |           |             |             |          |   |                                    |
| 16 | ("Skip") | ("Jump")                           |           |                              |           |             |             |          |   |                                    |
| 17 | SLE      | JLE                                | 2.1.2     | Jump Data <= Mask            | C         | R1          | R2          | 0        |   |                                    |
| 18 | SLT      | JLO                                | 2.1.3     | Jump Data < Mask             | C         | R1          | R2          | 1        |   |                                    |
| 19 | SE       | JEQ                                | 2.1.4     | Jump Data = Mask             | C         | R1          | R2          | 2        |   |                                    |
| 20 | SZ       | JNO                                | 2.1.1     | Jump No Data Bits Present    | C         | R1          | R2          | 3        |   |                                    |
| 21 | SS       | JALL                               | 2.1.5     | Jump All Data Bits Present   | C         | R1          | R2          | 4        |   |                                    |
| 22 | SBS      | JALLM                              | 2.1.6     | Jump Data = All Mask Bits    | C         | R1          | R2          | 5        |   |                                    |
| 23 | SBC      | JNOM                               | 2.1.7     | Jump Data = No Mask Bits     | C         | R1          | R2          | 6        |   |                                    |
| 24 | SBSH     | JHAM                               | 2.1.8     | Jump Data HI = All Mask Bits | C         | R1          | R2          | 7        |   |                                    |
| 25 | SGT      | JHI                                |           | NOT(<=) --> >                | C         | R1          | R2          | 8        |   | Corti: Skip if Great Than          |
| 26 | SGE      | JHE                                |           | NOT(<) --> >=                | C         | R1          | R2          | 9        |   | Corti: Skip if Great Than or Equal |
| 27 | SNE      | JHL                                |           | NOT(=) (Higher or Lower)     | C         | R1          | R2          | A        |   | Corti: Skip if Not Equal           |
| 28 | SNZ      | JSB                                |           | NOT (NoData) --> SomeData    | C         | R1          | R2          | B        |   | Corti: Skip if Not Zero            |
| 29 | SNS      | JSN                                |           | NOT (AllData) --> NoData     | C         | R1          | R2          | C        |   | Corti: Skip if None Set            |
| 30 | SNBS     | JSNM                               |           | NOT (AllMask)                | C         | R1          | R2          | D        |   |                                    |
| 31 | SNBC     | JSM                                |           | NOT (NoMask)                 | C         | R1          | R2          | E        |   |                                    |
| 32 | SNBSH    | JHSNM                              |           | NOT (Data HI = AllMask)      | C         | R1          | R2          | F        |   |                                    |
| 33 |          | MVM2                               | 2.2.1     | Move Halfword, Minus 2       | 0         | R2          | R1          | 0        |   |                                    |
| 34 |          | MVM1                               | 2.2.1     | Move Halfword, Minus 1       | 0         | R2          | R1          | 1        |   |                                    |
| 35 |          | MVP1                               | 2.2.1     | Move Halfword, Plus 1        | 0         | R2          | R1          | 2        |   |                                    |
| 36 |          | MVP2                               | 2.2.1     | Move Halfword, Plus 2        | 0         | R2          | R1          | 3        |   |                                    |

# P.A.L.M. INSTRUCTION SET

[illegible]

|     | HI  | LO  |
|-----|-----|-----|
| R0  | 0   | 700 |
| R1  | --- | --- |
| R2  | --- | --- |
| R3  | --- | --- |
| R4  | --- | --- |
| R5  | --- | --- |
| R6  | --- | --- |
| R7  | --- | --- |
| R8  | --- | --- |
| R9  | --- | --- |
| R10 | --- | --- |
| R11 | --- | --- |
| R12 | --- | --- |
| R13 | --- | --- |
| R14 | --- | --- |
| R15 | --- | --- |

|         |        | 1 | 2 | 3 | 4 | 5 | 6 |
|---------|--------|---|---|---|---|---|---|
|         |        | 1 | 2 | 3 | 4 | 5 | 6 |
| LINE 1  | 0x0200 |   |   |   |   |   |   |
| LINE 2  | 0x0240 |   |   |   |   |   |   |
| LINE 3  | 0x0280 |   |   |   |   |   |   |
| LINE 4  | 0x02C0 |   |   |   |   |   |   |
| LINE 5  | 0x0300 |   |   |   |   |   |   |
| LINE 6  | 0x0340 |   |   |   |   |   |   |
| LINE 7  | 0x0380 |   |   |   |   |   |   |
| LINE 8  | 0x03C0 |   |   |   |   |   |   |
| LINE 9  | 0x0400 |   |   |   |   |   |   |
| LINE 10 | 0x0440 |   |   |   |   |   |   |
| LINE 11 | 0x0480 |   |   |   |   |   |   |
| LINE 12 | 0x04C0 |   |   |   |   |   |   |
| LINE 13 | 0x0500 |   |   |   |   |   |   |
| LINE 14 | 0x0540 |   |   |   |   |   |   |
| LINE 15 | 0x0580 |   |   |   |   |   |   |
| LINE 16 | 0x05C0 |   |   |   |   |   |   |
|         | 0x0600 |   |   |   |   |   |   |

|     | HI   | LO |
|-----|------|----|
| R0  | PC   |    |
| R1  | ---- |    |
| R2  | ---- |    |
| R3  | ---- |    |
| R4  | ---- |    |
| R5  | ---- |    |
| R6  | ---- |    |
| R7  | ---- |    |
| R8  | ---- |    |
| R9  | 008D |    |
| R10 | 0000 |    |
| R11 | ---- |    |
| R12 | ---- |    |
| R13 | ---- |    |
| R14 | ---- |    |
| R15 | ---- |    |

|         |                                                                  |                                                                                    |   |   |   |   |   |   |
|---------|------------------------------------------------------------------|------------------------------------------------------------------------------------|---|---|---|---|---|---|
|         |                                                                  |                                                                                    | 1 | 2 | 3 | 4 | 5 | 6 |
|         | 1234567890123456789012345678901234567890123456789012345678901234 |                                                                                    |   |   |   |   |   |   |
| LINE 1  | 0x0200                                                           |  |   |   |   |   |   |   |
| LINE 2  | 0x0240                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 3  | 0x0280                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 4  | 0x02C0                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 5  | 0x0300                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 6  | 0x0340                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 7  | 0x0380                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 8  | 0x03C0                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 9  | 0x0400                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 10 | 0x0440                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 11 | 0x0480                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 12 | 0x04C0                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 13 | 0x0500                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 14 | 0x0540                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 15 | 0x0580                                                           |                                                                                    |   |   |   |   |   |   |
| LINE 16 | 0x05C0                                                           |                                                                                    |   |   |   |   |   |   |
|         | 0x0600                                                           |                                                                                    |   |   |   |   |   |   |

|      |      |      |      |      |                |               |
|------|------|------|------|------|----------------|---------------|
| 0700 | 0203 | D021 | 09F4 | CALL | ClrScr, R2     |               |
| 0706 | D901 | 0E00 |      | LWI  | R9, BANNER_LEN | → R9 = 0x0E00 |
| 070A | 6998 |      |      | LDBI | R9, R9         | → R9 = 141    |
| 070C | 8A00 |      |      | LBI  | R10, 0         | → Lo R10 = 0  |
| 070E | 0AAD |      |      | LTH  | R10, R10       | → Hi R10 = 0  |

|     | HI LO |
|-----|-------|
| R0  | PC    |
| R1  | ----  |
| R2  | ----  |
| R3  | ----  |
| R4  | ----  |
| R5  | ----  |
| R6  | ----  |
| R7  | ----  |
| R8  | ----  |
| R9  | 0000  |
| R10 | 0468  |
| R11 | ----  |
| R12 | ----  |
| R13 | ----  |
| R14 | ----  |
| R15 | ----  |

|         |        |                                                        |   |   |   |   |   |
|---------|--------|--------------------------------------------------------|---|---|---|---|---|
|         |        | 1                                                      | 2 | 3 | 4 | 5 | 6 |
|         |        | 123456789012345678901234567890123456789012345678901234 |   |   |   |   |   |
| LINE 1  | 0x0200 |                                                        |   |   |   |   |   |
| LINE 2  | 0x0240 |                                                        |   |   |   |   |   |
| LINE 3  | 0x0280 |                                                        |   |   |   |   |   |
| LINE 4  | 0x02C0 |                                                        |   |   |   |   |   |
| LINE 5  | 0x0300 |                                                        |   |   |   |   |   |
| LINE 6  | 0x0340 |                                                        |   |   |   |   |   |
| LINE 7  | 0x0380 |                                                        |   |   |   |   |   |
| LINE 8  | 0x03C0 |                                                        |   |   |   |   |   |
| LINE 9  | 0x0400 |                                                        |   |   |   |   |   |
| LINE 10 | 0x0440 |                                                        |   |   |   |   |   |
| LINE 11 | 0x0480 |                                                        |   |   |   |   |   |
| LINE 12 | 0x04C0 |                                                        |   |   |   |   |   |
| LINE 13 | 0x0500 |                                                        |   |   |   |   |   |
| LINE 14 | 0x0540 |                                                        |   |   |   |   |   |
| LINE 15 | 0x0580 |                                                        |   |   |   |   |   |
| LINE 16 | 0x05C0 |                                                        |   |   |   |   |   |
|         | 0x0600 |                                                        |   |   |   |   |   |

|                       |      |      |      |                      |               |
|-----------------------|------|------|------|----------------------|---------------|
| FIND_LEN_NEXT:        | 0710 | C903 | SZ   | R9                   |               |
|                       | 0712 | A001 | BRA  | FIND_LEN_BYTES       |               |
|                       | 0714 | A005 | BRA  | CALCULATE_NEXT_SETUP |               |
| FIND_LEN_BYTES:       | 0716 | AA07 | ADDI | R10, #8              | → R10 = R10+8 |
|                       | 0718 | 0991 | DEC  | R9                   | → R9 = R9-1   |
|                       | 071A | F00B | BRA  | FIND_LEN_NEXT        |               |
| CALCULATE_NEXT_SETUP: |      |      |      |                      |               |
|                       | 071C |      |      |                      |               |

**R10 = R9 \* 8 → 141 \* 8 = 1,128 (0x0468)**



|     | HI   | LO |
|-----|------|----|
| R0  | PC   |    |
| R1  | ---- |    |
| R2  | ---- |    |
| R3  | ---- |    |
| R4  | ---- |    |
| R5  | ---- |    |
| R6  | ---- |    |
| R7  | ---- |    |
| R8  | ---- |    |
| R9  | 0004 |    |
| R10 | 0468 |    |
| R11 | 1000 |    |
| R12 | 0C96 |    |
| R13 | 0CAA |    |
| R14 | ---- |    |
| R15 | ---- |    |

CALCULATE\_NEXT\_SETUP:

|      |                |      |                        |                  |
|------|----------------|------|------------------------|------------------|
| 071C | 09AC           | HTL  | R9, R10                | → Lo R9 = Hi R10 |
| 071E | DB01 1000      | LWI  | R11, BANNER_TARGET     |                  |
| 0722 | DC01 0C96      | LWI  | R12, WORKING_ADDRESS   |                  |
| 0726 | DD01 0CAA      | LWI  | R13, WORKING_ADDRESSI  |                  |
| 072A | 5BC1           | MOVE | (R12)+, R11            | ; ROW 1          |
| 072C | 5BD1           | MOVE | (R13)+, R11            |                  |
| 072E | 0203 D021 0A06 | CALL | IncreaseR11_by_Len, R2 |                  |
| 0734 | 5BC1           | MOVE | (R12)+, R11            | ; ROW 2          |
| 0736 | 5BD1           | MOVE | (R13)+, R11            |                  |
| 0738 | 0203 D021 0A06 | CALL | IncreaseR11_by_Len, R2 |                  |
| 073E | 5BC1           | MOVE | (R12)+, R11            | ; ROW 3          |
| 0740 | 5BD1           | MOVE | (R13)+, R11            |                  |
| 0742 | 0203 D021 0A06 | CALL | IncreaseR11_by_Len, R2 |                  |
| 0748 | 5BC1           | MOVE | (R12)+, R11            | ; ROW 4          |
| 074A | 5BD1           | MOVE | (R13)+, R11            |                  |
| 074C | 0203 D021 0A06 | CALL | IncreaseR11_by_Len, R2 |                  |

...

|     | HI   | LO |
|-----|------|----|
| R0  | PC   |    |
| R1  | ---- |    |
| R2  | ---- |    |
| R3  | ---- |    |
| R4  | ---- |    |
| R5  | ---- |    |
| R6  | ---- |    |
| R7  | ---- |    |
| R8  | ---- |    |
| R9  | 0004 |    |
| R10 | 0468 |    |
| R11 | 21A0 |    |
| R12 | 0C9E |    |
| R13 | 0CB2 |    |
| R14 | ---- |    |
| R15 | ---- |    |

CALCULATE\_NEXT\_SETUP:

```

071C    09AC    HTL    R9, R10
071E    DB01 1000    LWI    R11, BANNER_TARGET
0722    DC01 0C96    LWI    R12, WORKING_ADDRESS
0726    DD01 0CAA    LWI    R13, WORKING_ADDRESSI
072A    5BC1    MOVE    (R12)+, R11 ; ROW 1
072C    5BD1    MOVE    (R13)+, R11
072E    0203 D021 0A06    CALL    IncreaseR11_by_Len, R2
0734    5BC1    MOVE    (R12)+, R11 ; ROW 2
0736    5BD1    MOVE    (R13)+, R11
0738    0203 D021 0A06    CALL    IncreaseR11_by_Len, R2
073E    5BC1    MOVE    (R12)+, R11 ; ROW 3
0740    5BD1    MOVE    (R13)+, R11
0742    0203 D021 0A06    CALL    IncreaseR11_by_Len, R2
0748    5BC1    MOVE    (R12)+, R11 ; ROW 4
074A    5BD1    MOVE    (R13)+, R11
074C    0203 D021 0A06    CALL    IncreaseR11_by_Len, R2

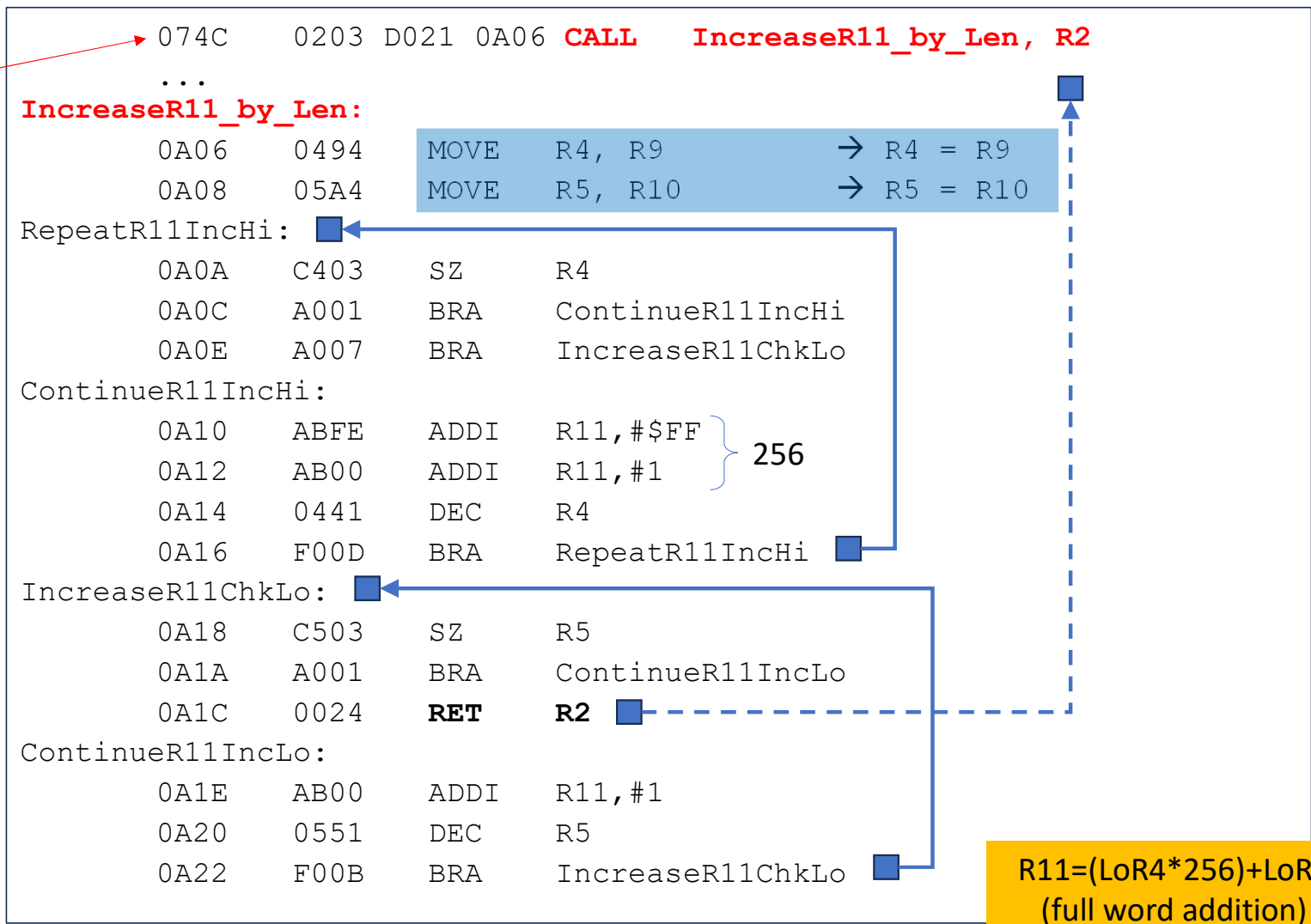
```

...

|      |      |
|------|------|
| 0C96 | 1000 |
| 0C98 | 1468 |
| 0C9A | 18D0 |
| 0C9C | 1D38 |
| 0C9E | ---- |
| 0CA0 | ---- |
| 0CA2 | ---- |
| 0CA4 | ---- |
| 0CA6 | ---- |
| 0CA8 | ---- |
| 0CAA | 1000 |
| 0CAC | 1468 |
| 0CAE | 18D0 |
| 0CB0 | 1D38 |
| 0CB2 | ---- |
| 0CB4 | ---- |
| 0CB6 | ---- |
| 0CB8 | ---- |
| 0CBA | ---- |
| 0CBC | ---- |



|     | HI   | LO |
|-----|------|----|
| R0  | 074C |    |
| R1  | ---- |    |
| R2  | ---- |    |
| R3  | ---- |    |
| R4  | 0004 |    |
| R5  | 0468 |    |
| R6  | ---- |    |
| R7  | ---- |    |
| R8  | ---- |    |
| R9  | 0004 |    |
| R10 | 0468 |    |
| R11 | 1D38 |    |
| R12 | 0C9E |    |
| R13 | 0CB2 |    |
| R14 | ---- |    |
| R15 | ---- |    |



|     | HI   | LO |
|-----|------|----|
| R0  | 078C |    |
| R1  | 0A1C |    |
| R2  | 0784 |    |
| R3  | 0E02 |    |
| R4  | 0000 |    |
| R5  | 0400 |    |
| R6  | ---- |    |
| R7  | ---- |    |
| R8  | ---- |    |
| R9  | 0004 |    |
| R10 | 0468 |    |
| R11 | 37A8 |    |
| R12 | 0CAA |    |
| R13 | 0CBE |    |
| R14 | ---- |    |
| R15 | ---- |    |

|                 |      |                |                             |        |
|-----------------|------|----------------|-----------------------------|--------|
| ...             | 0752 | 5BC1           | MOVE (R12)+, R11            | ROW 5  |
|                 | 0754 | 5BD1           | MOVE (R13)+, R11            |        |
|                 | 0756 | 0203 D021 0A06 | CALL IncreaseR11_by_Len, R2 |        |
|                 | 075C | 5BC1           | MOVE (R12)+, R11            | ROW 6  |
|                 | 075E | 5BD1           | MOVE (R13)+, R11            |        |
|                 | 0760 | 0203 D021 0A06 | CALL IncreaseR11_by_Len, R2 |        |
|                 | 0766 | 5BC1           | MOVE (R12)+, R11            | ROW 7  |
|                 | 0768 | 5BD1           | MOVE (R13)+, R11            |        |
|                 | 076A | 0203 D021 0A06 | CALL IncreaseR11_by_Len, R2 |        |
|                 | 0770 | 5BC1           | MOVE (R12)+, R11            | ROW 8  |
|                 | 0772 | 5BD1           | MOVE (R13)+, R11            |        |
|                 | 0774 | 0203 D021 0A06 | CALL IncreaseR11_by_Len, R2 |        |
|                 | 077A | 5BC1           | MOVE (R12)+, R11            | ROW 9  |
|                 | 077C | 5BD1           | MOVE (R13)+, R11            |        |
|                 | 077E | 0203 D021 0A06 | CALL IncreaseR11_by_Len, R2 |        |
|                 | 0784 | 5BC1           | MOVE (R12)+, R11            | ROW 10 |
|                 | 0786 | 5BD1           | MOVE (R13)+, R11            |        |
|                 | 0788 | D301 0E02      | LWI R3, BANNER_STRING       |        |
| CALCULATE NEXT: | 078C | ...            |                             |        |

|      |      |
|------|------|
| 0C96 | 1000 |
| 0C98 | 1468 |
| 0C9A | 18D0 |
| 0C9C | 1D38 |
| 0C9E | 21A0 |
| 0CA0 | 2608 |
| 0CA2 | 2A70 |
| 0CA4 | 2ED8 |
| 0CA6 | 3340 |
| 0CA8 | 37A8 |
| 0CAA | 1000 |
| 0CAC | 1468 |
| 0CAE | 18D0 |
| 0CB0 | 1D38 |
| 0CB2 | 21A0 |
| 0CB4 | 2608 |
| 0CB6 | 2A70 |
| 0CB8 | 2ED8 |
| 0CBA | 3340 |
| 0CBC | 37A8 |

|     | HI   | LO |
|-----|------|----|
| R0  | 0792 |    |
| R1  | 0A1C |    |
| R2  | 0784 |    |
| R3  | 0E04 |    |
| R4  | 0001 |    |
| R5  | 01BC |    |
| R6  | --FF |    |
| R7  | ---- |    |
| R8  | ---- |    |
| R9  | 0004 |    |
| R10 | 0468 |    |
| R11 | 37A8 |    |
| R12 | 0CAA |    |
| R13 | 0CBE |    |
| R14 | ---- |    |
| R15 | ---- |    |

...  
CALCULATE\_NEXT:

078C D531  
078E 045C  
0790 86FF  
0792 C642  
0794 A00B  
0796 86FF  
0798 C652  
079A A005  
079C 0203  
079E D021  
07A0 08F6

CALCULATE\_LARGE:

07A2 D601  
...

MOVE R5, (R3)+ → R5 = MEM[R3], R3=R3+2  
HTL R4, R5  
LBI R6, NULL\_TERM\_HI

SE R6, R4  
BRA CALCULATE\_LARGE  
LBI R6, NULL\_TERM\_LO  
SE R6, R5  
BRA CALCULATE\_LARGE  
CALL MAINLOOP\_INIT, R2

LWI R6, BIG\_LETTERS

|      |      |                  |
|------|------|------------------|
| 0E02 | 01BC | (header blank 1) |
| 0E04 | 01BC | (header blank 2) |
| 0E06 | 01BC | (header blank 3) |
| 0E08 | 01BC | (header blank 4) |
| 0E0A | 01BC | (header blank 5) |
| 0E0C | 01BC | (header blank 6) |
| 0E0E | 01BC | (header blank 7) |
| 0E10 | 01BC | (header blank 8) |
| 0E12 | 00FC | 252 V            |
| 0E14 | 0018 | 24 C             |
| 0E16 | 003C | 60 F             |
| 0E18 | 01BC | 444 " " (blank)  |
| 0E1A | 0024 | 36 D             |
| 0E1C | 0000 | 0 A              |
| 0E1E | 0084 | 132 L            |
| 0E20 | 0084 | 132 L            |
| 0E22 | 0000 | 0 A              |
| 0E24 | 00D8 | 216 S            |
| 0E26 | 01BC | 444 " " (blank)  |
| 0E28 | 0150 | 336 2            |
|      | ...  |                  |

|     | HI   | LO |
|-----|------|----|
| R0  | 07   | 92 |
| R1  | 0A   | 1C |
| R2  | 07   | 84 |
| R3  | 0E   | 04 |
| R4  | 00   | 01 |
| R5  | 01   | BC |
| R6  | --   | FF |
| R7  | ---- |    |
| R8  | ---- |    |
| R9  | 00   | 04 |
| R10 | 04   | 68 |
| R11 | 37   | A8 |
| R12 | 0C   | AA |
| R13 | 0C   | BE |
| R14 | ---- |    |
| R15 | ---- |    |

...  
CALCULATE\_NEXT:

078C D531 MOVE R5, (R3)+ → R5 = MEM[R3], R3=R3+2

078E 045C HTL R4, R5

0790 86FF LBI R6, NULL\_TERM\_HI

0792 C642 SE R6, R4 → Skip If Lo R6 = Lo R4 (Equal)

0794 A00B BRA CALCULATE\_LARGE

0796 86FF LBI R6, NULL\_TERM\_LO

0798 C652 SE R6, R5

079A A005 BRA CALCULATE\_LARGE

079C 0203 CALL MAINLOOP\_INIT, R2

079E D021

07A0 08F6

CALCULATE\_LARGE:

07A2 D601 LWI R6, BIG\_LETTERS

...

Else: since the LO is equal, then check the HI portion also. If both match, then we are done calculating and so, proceed to the main-loop portion.

|     | HI   | LO |
|-----|------|----|
| R0  | PC   |    |
| R1  | 0A1C |    |
| R2  | 0784 |    |
| R3  | 0E04 |    |
| R4  | 0001 |    |
| R5  | 01BC |    |
| R6  | 0A24 |    |
| R7  | ---- |    |
| R8  | ---- |    |
| R9  | 0004 |    |
| R10 | 0468 |    |
| R11 | 37A8 |    |
| R12 | 0CAA |    |
| R13 | 0CBE |    |
| R14 | ---- |    |
| R15 | ---- |    |

```

CALCULATE_LARGE:
    07A2    D601 0A24    LWI    R6, BIG_LETTERS

AGAIN_HI:
    07A6    C403    SZ    R4
    07A8    A001    BRA    EXHAUST_HI
    07AA    A007    BRA    AGAIN_LO

EXHAUST_HI:
    07AC    A6FE    ADDI    R6, #$FF
    07AE    A600    ADDI    R6, #1
    07B0    0441    DEC    R4
    07B2    F00D    BRA    AGAIN_HI

AGAIN_LO:
    07B4    C503    SZ    R5
    07B6    A001    BRA    EXHAUST_LO
    07B8    A005    BRA    PROCESS_ROWS

EXHAUST_LO:
    07BA    A600    ADDI    R6, 1
    07BC    0551    DEC    R5
    07BE    F00B    BRA    AGAIN_LO

PROCESS_ROWS:
    07C0    ...
  
```

$R6 = @BIG\_LETTERS + (Lo\ R4 * 256) + Lo\ R5$   
 $0x01BC\ (hex) == 444\ decimal$   
 (same 16-bit ADD as done in **IncreaseR11\_by\_Len**)

| BIG_LETTERS: |    |            |     |
|--------------|----|------------|-----|
| 0A24         | 01 | A          | 0   |
| 0A25         | 00 | (reserved) |     |
| 0A26         | 0C | 00001100   |     |
| 0A27         | 1E | 00011110   |     |
| 0A28         | 33 | 00110011   |     |
| 0A29         | 33 | 00110011   |     |
| 0A2A         | 3F | 00111111   |     |
| 0A2B         | 3F | 00111111   |     |
| 0A2C         | 33 | 00110011   |     |
| 0A2D         | 33 | 00110011   |     |
| 0A2E         | 33 | 00110011   |     |
| 0A2F         | 33 | 00110011   |     |
| 0A30         | 02 | B          | 12  |
| 0A31         | 00 | (reserved) |     |
| 0A32         | 3E | 00111110   |     |
| 0A33         | 3E | 00111110   |     |
| 0A34         | 33 | 00110011   |     |
| 0A35         | 33 | 00110011   |     |
| 0A36         | 3E | 00111110   |     |
| 0A37         | 3E | 00111110   |     |
| 0A38         | 33 | 00110011   |     |
| 0A39         | 33 | 00110011   |     |
| 0A3A         | 3E | 00111110   |     |
| 0A3B         | 3E | 00111110   |     |
| ...          |    |            |     |
| 0BE0         | 00 | " "(blank) | 444 |
| 0BE1         | 00 | (reserved) |     |
| 0BE2         | 00 | 00000000   |     |
| 0BE3         | 00 | 00000000   |     |
| 0BE4         | 00 | 00000000   |     |
| 0BE5         | 00 | 00000000   |     |
| 0BE6         | 00 | 00000000   |     |
| 0BE7         | 00 | 00000000   |     |
| 0BE8         | 00 | 00000000   |     |
| 0BE9         | 00 | 00000000   |     |
| 0BEA         | 00 | 00000000   |     |
| 0BEB         | 00 | 00000000   |     |

|     | HI   | LO |
|-----|------|----|
| R0  | 07C0 |    |
| R1  | 07B8 |    |
| R2  | 0784 |    |
| R3  | 0E04 |    |
| R4  | 0000 |    |
| R5  | 0100 |    |
| R6  | 0BE2 |    |
| R7  | 0000 |    |
| R8  | ---- |    |
| R9  | 0005 |    |
| R10 | 0468 |    |
| R11 | 37A8 |    |
| R12 | 0C96 |    |
| R13 | 0CBE |    |
| R14 | ---- |    |
| R15 | ---- |    |

#### PROCESS\_ROWS:

```

07C0 D761
07C2 077C
07C4 DC01 0C96
07C8 8905

```

```

MOVE R7, (R6)+ → R7 = MEM[R6], R6=R6+2
HTL  R7, R7    → Lo R7 = Hi R7
LWI  R12, WORKING_ADDRESS
LBI  R9, #$05  → Lo R9 = 05

```

#### PROCESS\_ROWS\_NEXT:

```

07CA DBC8
07CC D861
07CE 048C
07D0 8E80
07D2 C4ED
07D4 A005
07D6 8500
07D8 055D
07DA A001

```

```

MOVE R11, (R12)
MOVE R8, (R6)+
HTL  R4, R8
LBI  R14, #128
SNBS R4, R14
BRA  SET_COL1
LBI  R5, BG_SYMBOL
LTH  R5, R5
BRA  DO_COL2

```

#### SET\_COL1:

```
07DC 057D
```

```
LTH R5, R7
```

#### DO\_COL2:

```
07DE 048C
```

```
HTL R4, R8
```

#### WORKING ADDRESS:

|      |      |
|------|------|
| 0C96 | 1000 |
| 0C98 | 1468 |
| 0C9A | 18D0 |
| 0C9C | 1D38 |
| 0C9E | 21A0 |
| 0CA0 | 2608 |
| 0CA2 | 2A70 |
| 0CA4 | 2ED8 |
| 0CA6 | 3340 |
| 0CA8 | 37A8 |

5x counter of "double rows" will mean that 10 total rows get processed.

0xBE0 - 0xA24 = 0x1BC

#### BIG LETTERS:

|      |    |            |     |
|------|----|------------|-----|
| 0A24 | 01 | A          | 0   |
| 0A25 | 00 | (reserved) |     |
| 0A26 | 0C | 00001100   |     |
| 0A27 | 1E | 00011110   |     |
| 0A28 | 33 | 00110011   |     |
| 0A29 | 33 | 00110011   |     |
| 0A2A | 3F | 00111111   |     |
| 0A2B | 3F | 00111111   |     |
| 0A2C | 33 | 00110011   |     |
| 0A2D | 33 | 00110011   |     |
| 0A2E | 33 | 00110011   |     |
| 0A2F | 33 | 00110011   |     |
| ...  |    |            |     |
| 0BE0 | 00 | " "(blank) | 444 |
| 0BE1 | 00 | (reserved) |     |
| 0BE2 | 00 | 00000000   | #1  |
| 0BE3 | 00 | 00000000   |     |
| 0BE4 | 00 | 00000000   | #2  |
| 0BE5 | 00 | 00000000   |     |
| 0BE6 | 00 | 00000000   | #3  |
| 0BE7 | 00 | 00000000   |     |
| 0BE8 | 00 | 00000000   | #4  |
| 0BE9 | 00 | 00000000   |     |
| 0BEA | 00 | 00000000   | #5  |
| 0BEB | 00 | 00000000   |     |

|     | HI    | LO |
|-----|-------|----|
| R0  | PC    |    |
| R1  | 07B8  |    |
| R2  | 0784  |    |
| R3  | 0E04  |    |
| R4  | 0000  |    |
| R5  | 0000  |    |
| R6  | 0BE4  |    |
| R7  | 0000  |    |
| R8  | 0000  |    |
| R9  | 0005  |    |
| R10 | 0468  |    |
| R11 | 1000  |    |
| R12 | 0C96  |    |
| R13 | 0CB E |    |
| R14 | xx80  |    |
| R15 | ----  |    |

#### PROCESS\_ROWS:

```

07C0 D761 MOVE R7, (R6)+ → R7 = MEM[R6], R6=R6+2
07C2 077C HTL R7, R7 → Lo R7 = Hi R7
07C4 DC01 0C96 LWI R12, WORKING_ADDRESS
07C8 8905 LBI R9, #$05 → Lo R9 = 05

```

#### PROCESS\_ROWS\_NEXT:

```

07CA DBC8 MOVE R11, (R12) → R11 = MEM[R12]
07CC D861 MOVE R8, (R6)+ → R8=R6, R6=R6+2
07CE 048C HTL R4, R8 → Lo R4 = Hi R8
07D0 8E80 LBI R14, #128 → Lo R14 = 1000 0000
07D2 C4ED SNBS R4, R14 → Skip If Not Bit Set
07D4 A005 BRA SET_COL1
07D6 8500 LBI R5, BG_SYMBOL → Lo R5 = 00
07D8 055D LTH R5, R5 → Lo R5 = Hi R5
07DA A001 BRA DO_COL2

```

#### SET\_COL1:

```
07DC 057D LTH R5, R7
```

#### DO\_COL2:

```
07DE ...
```

#### WORKING ADDRESS:

|      |      |
|------|------|
| 0C96 | 1000 |
| 0C98 | 1468 |
| 0C9A | 18D0 |
| 0C9C | 1D38 |
| 0C9E | 21A0 |
| 0CA0 | 2608 |
| 0CA2 | 2A70 |
| 0CA4 | 2ED8 |
| 0CA6 | 3340 |
| 0CA8 | 37A8 |

#### BIG LETTERS:

|      |    |            |     |
|------|----|------------|-----|
| 0A24 | 01 | A          | 0   |
| 0A25 | 00 | (reserved) |     |
| 0A26 | 0C | 00001100   |     |
| 0A27 | 1E | 00011110   |     |
| 0A28 | 33 | 00110011   |     |
| 0A29 | 33 | 00110011   |     |
| 0A2A | 3F | 00111111   |     |
| 0A2B | 3F | 00111111   |     |
| 0A2C | 33 | 00110011   |     |
| 0A2D | 33 | 00110011   |     |
| 0A2E | 33 | 00110011   |     |
| 0A2F | 33 | 00110011   |     |
| ...  |    |            |     |
| 0BE0 | 00 | " "(blank) | 444 |
| 0BE1 | 00 | (reserved) |     |
| 0BE2 | 00 | 00000000   | #1  |
| 0BE3 | 00 | 00000000   |     |
| 0BE4 | 00 | 00000000   | #2  |
| 0BE5 | 00 | 00000000   |     |
| 0BE6 | 00 | 00000000   | #3  |
| 0BE7 | 00 | 00000000   |     |
| 0BE8 | 00 | 00000000   | #4  |
| 0BE9 | 00 | 00000000   |     |
| 0BEA | 00 | 00000000   | #5  |
| 0BEB | 00 | 00000000   |     |

Processed Column 1,  
now process Column 2.

|     | HI   | LO |
|-----|------|----|
| R0  | PC   |    |
| R1  | 07B8 |    |
| R2  | 0784 |    |
| R3  | 0E04 |    |
| R4  | 00C3 |    |
| R5  | 26xx |    |
| R6  | 0BE4 |    |
| R7  | 2626 |    |
| R8  | C3E1 |    |
| R9  | 0005 |    |
| R10 | 0468 |    |
| R11 | 1000 |    |
| R12 | 0C96 |    |
| R13 | 0CBE |    |
| R14 | xx80 |    |
| R15 | ---- |    |

#### PROCESS\_ROWS:

```

07C0 D761 MOVE R7, (R6)+ → R7 = MEM[R6], R6=R6+2
07C2 077C HTL R7, R7 → Lo R7 = Hi R7
07C4 DC01 0C96 LWI R12, WORKING_ADDRESS
07C8 8905 LBI R9, #05 → Lo R9 = 05

```

#### PROCESS\_ROWS\_NEXT:

```

07CA DBC8 MOVE R11, (R12) → R11 = MEM[R12]
07CC D861 MOVE R8, (R6)+ → R8=R6, R6=R6+2
07CE 048C HTL R4, R8 → Lo R4 = Hi R8
07D0 8E80 LBI R14, #128 → Lo R14 = 1000 0000
07D2 C4ED SNBS R4, R14 → Skip If Not Bit Set

```

```

07D4 A005 BRA SET_COL1
07D6 8500 LBI R5, BG_SYME
07D8 055D LTH R5, R5 → Lo R5 = Hi R5
07DA A001 BRA DO_COL2

```

#### SET\_COL1:

```
07DC 057D
```

#### DO\_COL2:

```
07DE ...
```

Processed Column 1,  
now process Column 2.

COLUMN 12345678

(0x1000) ROW 1: ++ \_\_\_\_ ++

(0x1468) ROW 2: +++ \_\_\_\_ +

R4: 1100 0011 (C3)  
R14: 1000 0000 (80)  
→ Lo R5 = Hi R5

#### WORKING\_ADDRESS:

|      |      |
|------|------|
| 0C96 | 1000 |
| 0C98 | 1468 |
| 0C9A | 18D0 |
| 0C9C | 1D38 |
| 0C9E | 21A0 |
| 0CA0 | 2608 |
| 0CA2 | 2A70 |
| 0CA4 | 2ED8 |
| 0CA6 | 3340 |
| 0CA8 | 37A8 |

#### EXAMPLE

#### IBM 5100

0 (00)  
32 (20)



#### BIG LETTERS:

|      |    |            |     |
|------|----|------------|-----|
| 0A24 | 01 | A          | 0   |
| 0A25 | 00 | (reserved) |     |
| 0A26 | 0C | 00001100   |     |
| 0A27 | 1E | 00011110   |     |
| 0A28 | 33 | 00110011   |     |
| 0A29 | 33 | 00110011   |     |
| 0A2A | 3F | 00111111   |     |
| 0A2B | 3F | 00111111   |     |
| 0A2C | 33 | 00110011   |     |
| 0A2D | 33 | 00110011   |     |
| 0A2E | 33 | 00110011   |     |
| 0A2F | 33 | 00110011   |     |
| ...  |    |            |     |
| 0BE0 | 26 | " "(blank) | 444 |
| 0BE1 | 00 | (reserved) |     |
| 0BE2 | C3 | 11000011   | #1  |
| 0BE3 | E1 | 11100001   | #1  |
| 0BE4 | 00 | 00000000   | #2  |
| 0BE5 | 00 | 00000000   | #2  |
| 0BE6 | 00 | 00000000   | #3  |
| 0BE7 | 00 | 00000000   | #3  |
| 0BE8 | 00 | 00000000   | #4  |
| 0BE9 | 00 | 00000000   | #4  |
| 0BEA | 00 | 00000000   | #5  |
| 0BEB | 00 | 00000000   | #5  |



|     | HI   | LO |
|-----|------|----|
| R0  | PC   |    |
| R1  | 07B8 |    |
| R2  | 0784 |    |
| R3  | 0E04 |    |
| R4  | 00C3 |    |
| R5  | 2626 |    |
| R6  | 0BE4 |    |
| R7  | 2626 |    |
| R8  | C3E1 |    |
| R9  | 0005 |    |
| R10 | 0468 |    |
| R11 | 1000 |    |
| R12 | 0C96 |    |
| R13 | 0CBE |    |
| R14 | xx40 |    |
| R15 | ---- |    |

|           |                   |                              |
|-----------|-------------------|------------------------------|
| DO_COL2:  |                   |                              |
| 07DE 048C | HTL R4, R8        | → Lo R4 = Hi R8              |
| 07E0 8E40 | LBI R14, #64      | → R14 = 0100 0000            |
| 07E2 C4ED | SNBS R4, R14      | → Skip If Not Bit Set        |
| 07E4 A005 | BRA SET_COL2      |                              |
| 07E6 8500 | LBI R5, BG_SYMBOL |                              |
| 07E8 55B1 | MOVE (R11)+, R5   |                              |
| 07EA A003 | BRA DO_COL3       |                              |
| SET_COL2: |                   |                              |
| 07EC 057C | HTL R5, R7        | → Lo R5 = Hi R7              |
| 07EE 55B1 | MOVE (R11)+, R5   | → MEM[R11] = R5, R11 = R11+2 |
| DO_COL3:  |                   |                              |
| 07F0 ...  |                   |                              |

R4: 1100 0011 (C3)  
R14: 0100 0000 (40)

## EXAMPLE

### IBM 5100

0 (00)  
32 (20)



### BIG LETTERS:

|      |    |            |     |
|------|----|------------|-----|
| 0A24 | 01 | A          | 0   |
| 0A25 | 00 | (reserved) |     |
| 0A26 | 0C | 00001100   |     |
| 0A27 | 1E | 00011110   |     |
| 0A28 | 33 | 00110011   |     |
| 0A29 | 33 | 00110011   |     |
| 0A2A | 3F | 00111111   |     |
| 0A2B | 3F | 00111111   |     |
| 0A2C | 33 | 00110011   |     |
| 0A2D | 33 | 00110011   |     |
| 0A2E | 33 | 00110011   |     |
| 0A2F | 33 | 00110011   |     |
| ...  |    |            |     |
| 0BE0 | 26 | " "(blank) | 444 |
| 0BE1 | 00 | (reserved) |     |
| 0BE2 | C3 | 11000011   | #1  |
| 0BE3 | E1 | 11100001   |     |
| 0BE4 | 00 | 00000000   | #2  |
| 0BE5 | 00 | 00000000   |     |
| 0BE6 | 00 | 00000000   | #3  |
| 0BE7 | 00 | 00000000   |     |
| 0BE8 | 00 | 00000000   | #4  |
| 0BE9 | 00 | 00000000   |     |
| 0BEA | 00 | 00000000   | #5  |
| 0BEB | 00 | 00000000   |     |

Processed Column 2, commit  
column 1 and 2 to memory.  
Then process Column 3 and 4.

### WORKING ADDRESS:

|      |      |
|------|------|
| 0C96 | 1000 |
| 0C98 | 1468 |
| 0C9A | 18D0 |
| 0C9C | 1D38 |
| 0C9E | 21A0 |
| 0CA0 | 2608 |
| 0CA2 | 2A70 |
| 0CA4 | 2ED8 |
| 0CA6 | 3340 |
| 0CA8 | 37A8 |

COLUMN 12345678

(0x1000) ROW 1: ++ \_\_\_\_ ++  
(0x1468) ROW 2: +++ \_\_\_\_ +

|     | HI           | LO |
|-----|--------------|----|
| R0  | 085E         |    |
| R1  | 07B8         |    |
| R2  | 0784         |    |
| R3  | 0E04         |    |
| R4  | <b>C3E1</b>  |    |
| R5  | <b>2626</b>  |    |
| R6  | <b>0BE4</b>  |    |
| R7  | <b>2626</b>  |    |
| R8  | <b>C3E1</b>  |    |
| R9  | 0005         |    |
| R10 | 0468         |    |
| R11 | <b>1468</b>  |    |
| R12 | 0C98         |    |
| R13 | 0CBE         |    |
| R14 | xx01         |    |
| R15 | xx <b>80</b> |    |

SET\_COL8:

0852 057C

HTL R5, R7 → Lo R5 = Hi R7

0854 55B1

MOVE (R11)+, R5 → MEM[R11]=R5, R11=R11+2

DONE\_COLUMNS\_FIRST\_ROW:

0856 5BC1

MOVE (R12)+, R11 → MEM[R12]=R11, R12=R12+2

0858 DBC8

MOVE R11, (R12) → R11 = MEM[R12]

DO\_COL1\_R2: ; R2 == ROW2 (Lo PORTION OF R8)

085A 0484

MOVE R4, R8 → R4 = R8

085C 8F80

LBI R15, #128 → Lo R15 = 0x80

085E C4FD

SNBS R4, R15

0860 A005

BRA SET\_COL1\_R2

0862 8500

LBI R5, BG\_SYMBOL

0864 055D

LTH R5, R5

0866 A001

BRA DO\_COL2\_R2

SET\_COL1\_R2:

0868 057D

LTH R5, R7

DO\_COL2\_R2:

086A 0484

MOVE R4, R8

086C 8F40

LBI R15, #64

WORKING ADDRESS:

|      |             |
|------|-------------|
| 0C96 | <b>1008</b> |
| 0C98 | 1468        |
| 0C9A | 18D0        |
| 0C9C | 1D38        |
| 0C9E | 21A0        |
| 0CA0 | 2608        |
| 0CA2 | 2A70        |
| 0CA4 | 2ED8        |
| 0CA6 | 3340        |
| 0CA8 | 37A8        |

COLUMN **12345678**

(0x1000) ROW 1: ++      ++

(0x1468) ROW 2: +++      +

EXAMPLE

IBM 5100

0 (00)

32 (20)



BIG LETTERS:

|      |           |                 |     |
|------|-----------|-----------------|-----|
| 0A24 | 01        | A               | 0   |
| 0A25 | 00        | (reserved)      |     |
| 0A26 | 0C        | 00001100        |     |
| 0A27 | 1E        | 00011110        |     |
| 0A28 | 33        | 00110011        |     |
| 0A29 | 33        | 00110011        |     |
| 0A2A | 3F        | 00111111        |     |
| 0A2B | 3F        | 00111111        |     |
| 0A2C | 33        | 00110011        |     |
| 0A2D | 33        | 00110011        |     |
| 0A2E | 33        | 00110011        |     |
| 0A2F | 33        | 00110011        |     |
| ...  |           |                 |     |
| 0BE0 | <b>26</b> | " "(blank)      | 444 |
| 0BE1 | 00        | (reserved)      |     |
| 0BE2 | <b>C3</b> | <b>11000011</b> | #1  |
| 0BE3 | <b>E1</b> | <b>11100001</b> |     |
| 0BE4 | 00        | 00000000        | #2  |
| 0BE5 | 00        | 00000000        |     |
| 0BE6 | 00        | 00000000        | #3  |
| 0BE7 | 00        | 00000000        |     |
| 0BE8 | 00        | 00000000        | #4  |
| 0BE9 | 00        | 00000000        |     |
| 0BEA | 00        | 00000000        | #5  |
| 0BEB | 00        | 00000000        |     |

|     | HI   | LO |
|-----|------|----|
| R0  | 08EA |    |
| R1  | 08B6 |    |
| R2  | 08F6 |    |
| R3  | 0E14 |    |
| R4  | 0C0C |    |
| R5  | 0000 |    |
| R6  | 0B2C |    |
| R7  | 1616 |    |
| R8  | 0C0C |    |
| R9  | 0000 |    |
| R10 | 0468 |    |
| R11 | 37F0 |    |
| R12 | 0CAA |    |
| R13 | 0CBE |    |
| R14 | xx01 |    |
| R15 | xx01 |    |

```

SET_COL8_R2:
    08DE 057C HTL R5, R7
    08E0 55B1 MOVE (R11)+, R5
DONE_COLUMNS_NEXT_ROW:
    08E2 5BC1 MOVE (R12)+, R11 → MEM[R12]=R11, R12=R12+2
    08E4 0991 DEC R9 → D9=D9-1
    08E6 C903 SZ R9 → Skip if R9 is Zero
    08E8 A005 BRA PROCESS_ROWS_BIGJUMP
    08EA 0203 CALL CALCULATE_NEXT, R2 → (JMP BROKE)
    08EC D021
    08EE 078C
PROCESS_ROWS_BIGJUMP:
    08F0 0203 CALL PROCESS_ROWS_NEXT, R2 → (JMP BROKE)
    08F2 D021
    08F4 07CA
MAINLOOP_INIT:
    08F6 09A4 MOVE R9, R10

```

Calculate next letter  
in the banner string.

If R9 was not yet 0, then  
process next pair of rows  
in the current big-letter.

WORKING ADDRESS:

|      |      |
|------|------|
| 0C96 | 1048 |
| 0C98 | 14B0 |
| 0C9A | 1918 |
| 0C9C | 1D80 |
| 0C9E | 21E8 |
| 0CA0 | 2650 |
| 0CA2 | 2AB8 |
| 0CA4 | 2F20 |
| 0CA6 | 3388 |
| 0CA8 | 37F0 |

BIG LETTERS:

|      |    |            |     |
|------|----|------------|-----|
| 0A24 | 01 | A          | 0   |
| 0A25 | 00 | (reserved) |     |
| 0A26 | 0C | 00001100   |     |
| 0A27 | 1E | 00011110   |     |
| 0A28 | 33 | 00110011   |     |
| 0A29 | 33 | 00110011   |     |
| 0A2A | 3F | 00111111   |     |
| 0A2B | 3F | 00111111   |     |
| 0A2C | 33 | 00110011   |     |
| 0A2D | 33 | 00110011   |     |
| 0A2E | 33 | 00110011   |     |
| 0A2F | 33 | 00110011   |     |
| ...  |    |            |     |
| 0B20 | 16 | V          | 252 |
| 0B21 | 00 | (reserved) |     |
| 0B22 | 33 | 00110011   | #1  |
| 0B23 | 33 | 00110011   |     |
| 0B24 | 33 | 00110011   | #2  |
| 0B25 | 33 | 00110011   |     |
| 0B26 | 33 | 00110011   | #3  |
| 0B27 | 1F | 00011111   |     |
| 0B28 | 1E | 00011110   | #4  |
| 0B29 | 1E | 00011110   |     |
| 0B2A | 0C | 00001100   | #5  |
| 0B2B | 0C | 00001100   |     |

COLUMN 12345678

(0x3380) ROW 9:     VV    

(0x37E8) ROW10:     VV

1 2 3 4 5 6

1234567890123456789012345678901234567890123456789012345678901234

```

LINE 1  0x0200
LINE 2  0x0240
LINE 3  0x0280
LINE 4  0x02C0
LINE 5  0x0300
LINE 6  0x0340
LINE 7  0x0380
LINE 8  0x03C0
LINE 9  0x0400
LINE 10 0x0440
LINE 11 0x0480
LINE 12 0x04C0
LINE 13 0x0500
LINE 14 0x0540
LINE 15 0x0580
LINE 16 0x05C0
0x0600

```

|        |        |                    |        |
|--------|--------|--------------------|--------|
| ROW 1  | 0x1000 | VV_VV_CCC_FFFFFF   | 0x1058 |
| ROW 2  | 0x1468 | VV_VV_CCCCC_FFFFFF | 0x14C0 |
| ROW 3  | 0x18D0 | VV_VV_CCC_CC_FF    | 0x1928 |
| ROW 4  | 0x1D38 | VV_VV_CC_FF        | 0x1D90 |
| ROW 5  | 0x21A0 | VV_VV_CC_FFFF      | 0x21F8 |
| ROW 6  | 0x2608 | VVVV_CC_FFFF       | 0x2660 |
| ROW 7  | 0x2A70 | VVVV_CC_FF         | 0x2AC8 |
| ROW 8  | 0x2ED8 | VVVV_CCC_CC_FF     | 0x2F30 |
| ROW 9  | 0x3340 | VV_CCCCC_FF        | 0x3398 |
| ROW 10 | 0x37A8 | VV_CCC_FF          | 0x3800 |

"High RAM" general storage  
copied over to the 0x0200  
addresses that are memory-mapped  
to the CRT display.

LINE 1 0x0200  
LINE 2 0x0240  
LINE 3 0x0280  
LINE 4 0x02C0  
LINE 5 0x0300  
LINE 6 0x0340  
LINE 7 0x0380  
LINE 8 0x03C0  
LINE 9 0x0400  
LINE 10 0x0440  
LINE 11 0x0480  
LINE 12 0x04C0  
LINE 13 0x0500  
LINE 14 0x0540  
LINE 15 0x0580  
LINE 16 0x05C0  
0x0600

(initial)

ROW 1 0x1000  
ROW 2 0x1468  
ROW 3 0x18D0  
ROW 4 0x1D38  
ROW 5 0x21A0  
ROW 6 0x2608  
ROW 7 0x2A70  
ROW 8 0x2ED8  
ROW 9 0x3340  
ROW 10 0x37A8

(working)

1 2 3 4 5 6  
12345678901234567890123456789012345678901234



0x1000  
0x1468  
0x18D0  
0x1D38  
0x21A0  
0x2608  
0x2A70  
0x2ED8  
0x3340  
0x37A8

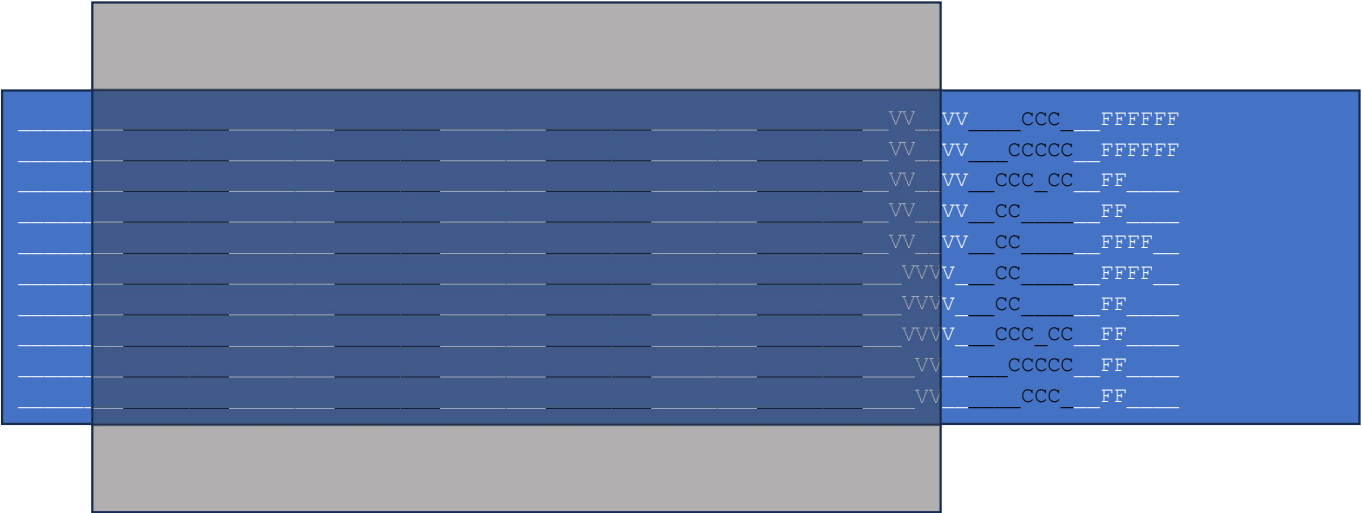
LINE 1 0x0200  
LINE 2 0x0240  
LINE 3 0x0280  
LINE 4 0x02C0  
LINE 5 0x0300  
LINE 6 0x0340  
LINE 7 0x0380  
LINE 8 0x03C0  
LINE 9 0x0400  
LINE 10 0x0440  
LINE 11 0x0480  
LINE 12 0x04C0  
LINE 13 0x0500  
LINE 14 0x0540  
LINE 15 0x0580  
LINE 16 0x05C0  
0x0600

(initial)

ROW 1 0x1000  
ROW 2 0x1468  
ROW 3 0x18D0  
ROW 4 0x1D38  
ROW 5 0x21A0  
ROW 6 0x2608  
ROW 7 0x2A70  
ROW 8 0x2ED8  
ROW 9 0x3340  
ROW 10 0x37A8

(working)

1 2 3 4 5 6  
12345678901234567890123456789012345678901234



0x1006  
0x146E  
0x18D6  
0x1D3E  
0x21A6  
0x260E  
0x2A76  
0x2EDE  
0x3346  
0x37AE

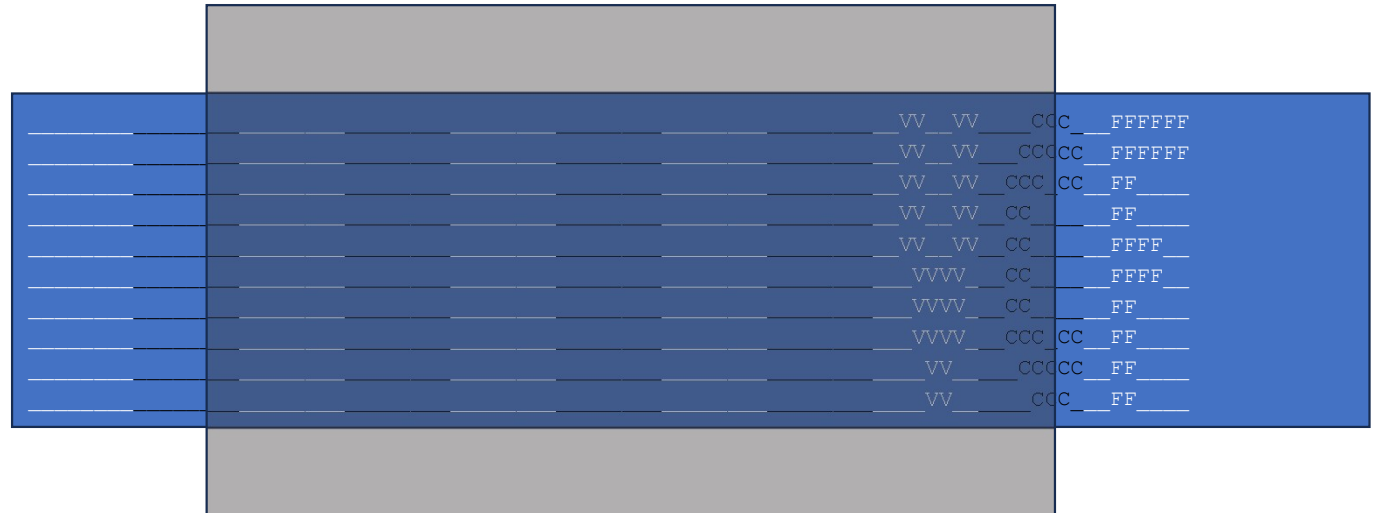
LINE 1 0x0200  
LINE 2 0x0240  
LINE 3 0x0280  
LINE 4 0x02C0  
LINE 5 0x0300  
LINE 6 0x0340  
LINE 7 0x0380  
LINE 8 0x03C0  
LINE 9 0x0400  
LINE 10 0x0440  
LINE 11 0x0480  
LINE 12 0x04C0  
LINE 13 0x0500  
LINE 14 0x0540  
LINE 15 0x0580  
LINE 16 0x05C0  
0x0600

(initial)

ROW 1 0x1000  
ROW 2 0x1468  
ROW 3 0x18D0  
ROW 4 0x1D38  
ROW 5 0x21A0  
ROW 6 0x2608  
ROW 7 0x2A70  
ROW 8 0x2ED8  
ROW 9 0x3340  
ROW 10 0x37A8

(working)

1 2 3 4 5 6  
12345678901234567890123456789012345678901234



0x100E  
0x1476  
0x18DE  
0x1D46  
0x21A6  
0x21AE  
0x2A7E  
0x2EE6  
0x334E  
0x37B6

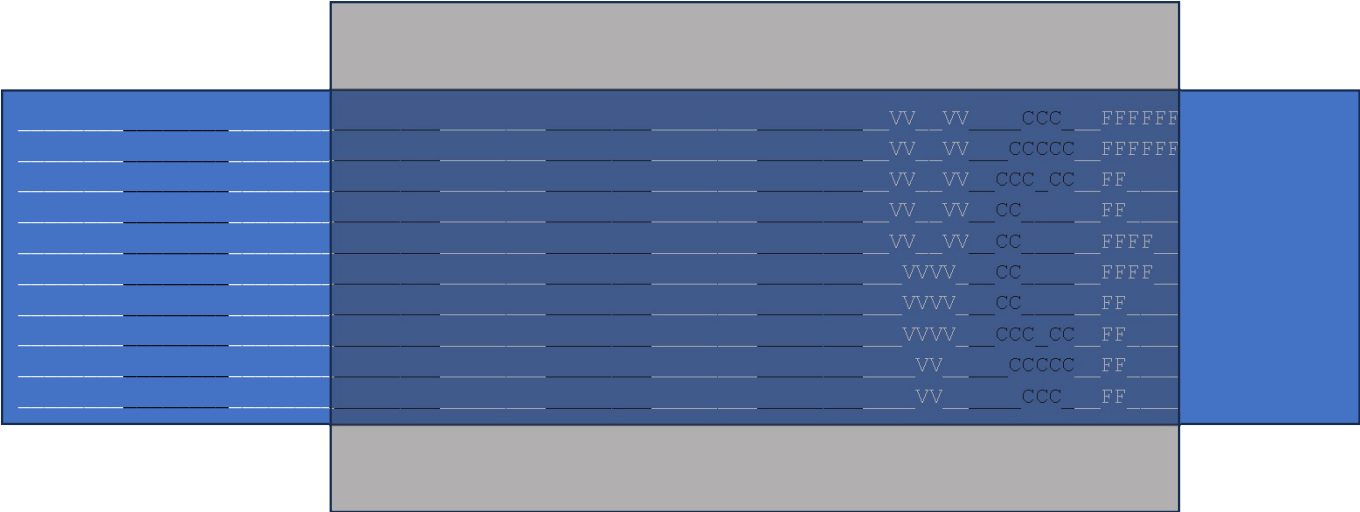
LINE 1 0x0200  
LINE 2 0x0240  
LINE 3 0x0280  
LINE 4 0x02C0  
LINE 5 0x0300  
LINE 6 0x0340  
LINE 7 0x0380  
LINE 8 0x03C0  
LINE 9 0x0400  
LINE 10 0x0440  
LINE 11 0x0480  
LINE 12 0x04C0  
LINE 13 0x0500  
LINE 14 0x0540  
LINE 15 0x0580  
LINE 16 0x05C0  
0x0600

(initial)

ROW 1 0x1000  
ROW 2 0x1468  
ROW 3 0x18D0  
ROW 4 0x1D38  
ROW 5 0x21A0  
ROW 6 0x2608  
ROW 7 0x2A70  
ROW 8 0x2ED8  
ROW 9 0x3340  
ROW 10 0x37A8

(working)

1 2 3 4 5 6  
12345678901234567890123456789012345678901234



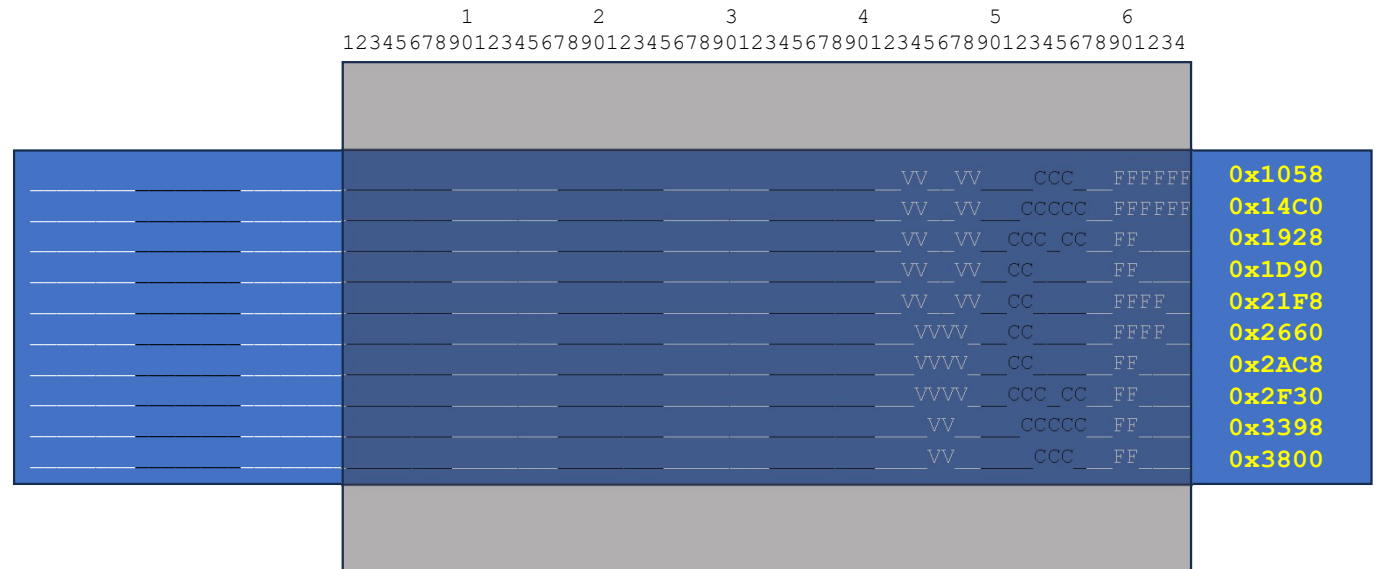
0x1018  
0x1480  
0x18E8  
0x1D50  
0x21B0  
0x21B8  
0x2A88  
0x2EF0  
0x3358  
0x37C0



LINE 1 0x0200  
 LINE 2 0x0240  
 LINE 3 0x0280  
 LINE 4 0x02C0  
 LINE 5 0x0300  
 LINE 6 0x0340  
 LINE 7 0x0380  
 LINE 8 0x03C0  
 LINE 9 0x0400  
 LINE 10 0x0440  
 LINE 11 0x0480  
 LINE 12 0x04C0  
 LINE 13 0x0500  
 LINE 14 0x0540  
 LINE 15 0x0580  
 LINE 16 0x05C0  
 0x0600

(initial)

ROW 1 0x1000  
 ROW 2 0x1468  
 ROW 3 0x18D0  
 ROW 4 0x1D38  
 ROW 5 0x21A0  
 ROW 6 0x2608  
 ROW 7 0x2A70  
 ROW 8 0x2ED8  
 ROW 9 0x3340  
 ROW 10 0x37A8



Should scroll increment by 1 byte or 2 byte?

Decided to go with 2 byte - easier and less code.

But will it look "jittery" ?

|     | HI   | LO |
|-----|------|----|
| R0  | 079C |    |
| R1  | 08EA |    |
| R2  | 08F0 |    |
| R3  | 0F1E |    |
| R4  | 00FF |    |
| R5  | FFFF |    |
| R6  | 0BFF |    |
| R7  | 0000 |    |
| R8  | 0000 |    |
| R9  | 0000 |    |
| R10 | 0468 |    |
| R11 | 3C10 |    |
| R12 | 0CAA |    |
| R13 | 0CBE |    |
| R14 | xx01 |    |
| R15 | xx01 |    |

CALCULATE\_NEXT:

078C D531

078E 045C

0790 86FF

0792 C642

0794 A00B

0796 86FF

0798 C652

079A A005

079C 0203

079E D021

07A0 08F6

CALCULATE\_LARGE:

07A2 ...

MOVE R5, (R3)+

HTL R4, R5

LBI R6, NULL\_TERM\_HI

SE R6, R4

BRA CALCULATE\_LARGE

LBI R6, NULL\_TERM\_LO

SE R6, R5

BRA CALCULATE\_LARGE

CALL MAINLOOP\_INIT, R2 ; (JMP BROKE)

...

|      |      |                      |   |
|------|------|----------------------|---|
| 0F08 | 00A8 | 168                  | 0 |
| 0F0A | 00A8 | 168                  | 0 |
| 0F0C | 01BC | (footer blank 1)     |   |
| 0F0E | 01BC | (footer blank 2)     |   |
| 0F10 | 01BC | (footer blank 3)     |   |
| 0F12 | 01BC | (footer blank 4)     |   |
| 0F14 | 01BC | (footer blank 5)     |   |
| 0F16 | 01BC | (footer blank 6)     |   |
| 0F18 | 01BC | (footer blank 7)     |   |
| 0F1A | 01BC | (footer blank 8)     |   |
| 0F1C | FFFF | END OF STRING marker |   |
| 0F1E | FFFF |                      |   |

|     | HI   | LO |
|-----|------|----|
| R0  | 090E |    |
| R1  | 08EA |    |
| R2  | 07A2 |    |
| R3  | 0F1E |    |
| R4  | 0CBE |    |
| R5  | 0D80 |    |
| R6  | 0BFF |    |
| R7  | 0000 |    |
| R8  | 0002 |    |
| R9  | 0414 |    |
| R10 | 0468 |    |
| R11 | 3C10 |    |
| R12 | 0CAA |    |
| R13 | 0CBE |    |
| R14 | 0601 |    |
| R15 | 1201 |    |

MAINLOOP\_INIT:

08F6 09A4

08F8 F93F

08FA 8800

08FC 088D

08FE 089C

0900 E08C

0902 E09C

0904 D401

0906 0CBE

0908 D501

090A 0D80

090C 5458

MAINLOOP\_INIT2:

090E ...

MOVE R9, R10 → R9 = R10 (copy length)

SUBI R9, #64 → R9 = R9 - 64 (sub. 64)

LBI R8, 0 → Lo R8 = 0 (split to R8/R9)

LTH R8, R8 → Hi R8 = Lo R8

HTL R8, R9 → Lo R8 = Hi R9

SHR R8 → Lo R8 = (Lo R8 >> 1)

SHR R9 → Lo R9 = (Lo R9 >> 1)

LWI R4, TITLE\_STRING (shift Hi & Lo)

LWI R5, TITLE\_STRING\_TEMP

MOVE (R5), R4 → MEM[R5] = R4

0x0468 = 0000 0100 0110 1000 ( R9)  
 0000 0100 0010 1000 = 0x042B (- 64)  
 0000 0010 0001 0100 = 0x0214  
 (R8 >> 1) (R9 >> 1)

To avoid scrolling past the last edge of the big letters, we need to decide how many “scroll steps” is appropriate for this banner.

We take the total number of expanded per-row length, subtract one screen width of columns (64 for this system), then divide by two since we are stepping 2-bytes per scroll.

**STEPS = (LEN - 64) / 2**

SHIFT-RIGHT operation used to divide by 2.

|     | HI   | LO |
|-----|------|----|
| R0  | 090E |    |
| R1  | 08EA |    |
| R2  | 07A2 |    |
| R3  | 0F1E |    |
| R4  | 0CBE |    |
| R5  | 0D80 |    |
| R6  | 0BFF |    |
| R7  | 0000 |    |
| R8  | 0002 |    |
| R9  | 0414 |    |
| R10 | 0468 |    |
| R11 | 3C10 |    |
| R12 | 0CAA |    |
| R13 | 0CBE |    |
| R14 | 0601 |    |
| R15 | 1201 |    |

|                 |      |                                        |
|-----------------|------|----------------------------------------|
| MAINLOOP_INIT:  |      |                                        |
| 08F6            | 09A4 | MOVE R9, R10 → R9 = R10 (copy length)  |
| 08F8            | F93F | SUBI R9, #64 → R9 = R9 - 64 (sub. 64)  |
| 08FA            | 8800 | LBI R8, 0 → Lo R8 = 0 (split to R8/R9) |
| 08FC            | 088D | LTH R8, R8 → Hi R8 = Lo R8             |
| 08FE            | 089C | HTL R8, R9 → Lo R8 = Hi R9             |
| 0900            | E08C | SHR R8 → Lo R8 = (Lo R8 >> 1)          |
| 0902            | E09C | SHR R9 → Lo R9 = (Lo R9 >> 1)          |
| 0904            | D401 | LWI R4, TITLE_STRING (shift Hi & Lo)   |
| 0906            | 0CBE |                                        |
| 0908            | D501 | LWI R5, TITLE_STRING_TEMP              |
| 090A            | 0D80 |                                        |
| 090C            | 5458 | MOVE (R5), R4 → MEM[R5] = R4           |
| MAINLOOP_INIT2: |      |                                        |
| 090E            | ...  |                                        |

We initialize TITLE\_STRING\_TEMP to the initial value of the @TITLE\_STRING (address). At the end of its scrolling, TITLE\_STRING\_TEMP will get reset back to @TITLE\_STRING.

|     | HI | LO |
|-----|----|----|
| R0  | 09 | 1C |
| R1  | 08 | EA |
| R2  | 07 | A2 |
| R3  | 00 | 00 |
| R4  | 0C | 96 |
| R5  | 0C | AA |
| R6  | 0B | FF |
| R7  | 00 | 0A |
| R8  | 00 | 02 |
| R9  | 04 | 14 |
| R10 | 04 | 68 |
| R11 | 3C | 10 |
| R12 | 0C | AA |
| R13 | 0C | BE |
| R14 | 06 | 01 |
| R15 | 12 | 01 |

MAINLOOP\_INIT2:

090E 8300

0910 033D

0912 D401

0914 0C96

0916 D501

0918 0CAA

091A 870A

REPEAT\_RESET:

091C ...

LBI R3, #0 → Lo R3 = 0

LTH R3, R3 → Hi R3 = Lo R3

LWI R4, WORKING\_ADDRESS → R4 = @WORK\_ADDR

LWI R5, WORKING\_ADDRESSI → R5 = @WORK\_ADDRI

LBI R7, #10 → Lo R7 = 10 (decimal)

**R3** will be used as our scene "step counter" to monitor for when we've reached near the edge.

**R4** points to the array of addresses for each of the big\_letter rows (used for the copy made on next page).

**R7** is a loop counter to keep count of how many rows we have copied so far (counting down from 10).

|     | HI   | LO |
|-----|------|----|
| R0  | 092E |    |
| R1  | 0926 |    |
| R2  | 07A2 |    |
| R3  | 0000 |    |
| R4  | 0CBE |    |
| R5  | 0CBE |    |
| R6  | 37A8 |    |
| R7  | 0000 |    |
| R8  | 0002 |    |
| R9  | 0414 |    |
| R10 | 0480 |    |
| R11 | 3C10 |    |
| R12 | 0CAA |    |
| R13 | 0CBE |    |
| R14 | 0601 |    |
| R15 | 1201 |    |

REPEAT\_RESET:

091C D651  
091E 5641  
0920 0771  
0922 C703  
0924 F009  
0926 D401  
0928 0D80  
092A D448  
092C 8A80

MAINLOOP:

092E 860A  
0930 DE01  
0932 0280  
0934 DD01  
0936 0C96

MAINLOOP\_A:

0938 ...

MOVE R6, (R5)+ → R6 = MEM[R5], R5=R5+2  
MOVE (R4)+, R6 → MEM[R4] = R6, R4=R4+2  
DEC R7 → R7 = R7-1  
SZ R7 → Skip if (R7 == 0)  
BRA REPEAT\_RESET  
LWI R4, TITLE\_STRING\_TEMP  
  
MOVE R4, (R4) → R4 = MEM[R4]  
LBI R10, #128 → Lo R10 = 0x80

Reset all 10 entries of WORKING\_ADDRESS back to the starting values stored in the WORKING\_ADDRESS\_I array.

|      |      |
|------|------|
| 0C96 | 1000 |
| 0C98 | 1468 |
| 0C9A | 18D0 |
| 0C9C | 1D38 |
| 0C9E | 21A0 |
| 0CA0 | 2608 |
| 0CA2 | 2A70 |
| 0CA4 | 2ED8 |
| 0CA6 | 3340 |
| 0CA8 | 37A8 |
| 0CAA | 1000 |
| 0CAC | 1468 |
| 0CAE | 18D0 |
| 0CB0 | 1D38 |
| 0CB2 | 21A0 |
| 0CB4 | 2608 |
| 0CB6 | 2A70 |
| 0CB8 | 2ED8 |
| 0CBA | 3340 |
| 0CBC | 37A8 |

**R4** now restored to where we were in the single row "title string" being scrolled.

**R10** is used to determine when to "reset" the single row "title string" scroll. We scroll the main string (64 chars) and then 64-blanks (so 128 steps).

|     | HI | LO |
|-----|----|----|
| R0  | 09 | 38 |
| R1  | 09 | 26 |
| R2  | 07 | A2 |
| R3  | 00 | 00 |
| R4  | 0C | BE |
| R5  | 0C | BE |
| R6  | 37 | 0A |
| R7  | 00 | 00 |
| R8  | 00 | 02 |
| R9  | 04 | 14 |
| R10 | 04 | 80 |
| R11 | 3C | 10 |
| R12 | 0C | AA |
| R13 | 0C | 96 |
| R14 | 02 | 80 |
| R15 | 12 | 01 |

```

REPEAT_RESET:
    091C D651 MOVE R6, (R5)+    → R6 = MEM[R5], R5=R5+2
    091E 5641 MOVE (R4)+, R6    → MEM[R4] = R6, R4=R4+2
    0920 0771 DEC R7            → R7 = R7-1
    0922 C703 SZ R7             → Skip if (R7 == 0)
    0924 F009 BRA REPEAT_RESET
    0926 D401 LWI R4, TITLE_STRING_TEMP
    0928 0D80
    092A D448 MOVE R4, (R4)      → R4 = MEM[R4]
    092C 8A80 LBI R10, #128     → Lo R10 = 0x80

MAINLOOP:
    092E 860A LBI R6, #10
    0930 DE01 LWI R14, FIRST_BANNER_ROW
    0932 0280
    0934 DD01 LWI R13, WORKING_ADDRESS
    0936 0C96

MAINLOOP_A:
    0938 ...
  
```

Reset all 10 entries of **WORKING\_ADDRESS** back to the starting values stored in the **WORKING\_ADDRESS\_I** array.

|      |      |
|------|------|
| 0C96 | 1000 |
| 0C98 | 1468 |
| 0C9A | 18D0 |
| 0C9C | 1D38 |
| 0C9E | 21A0 |
| 0CA0 | 2608 |
| 0CA2 | 2A70 |
| 0CA4 | 2ED8 |
| 0CA6 | 3340 |
| 0CA8 | 37A8 |

**R6** now used as 10-iteration loop counter (number of rows).

**R13** is first entry of the addresses used for each row.

**R14** is starting row on the CRT screen of where to output.

|     | HI | LO |
|-----|----|----|
| R0  | 09 | 3C |
| R1  | 09 | 26 |
| R2  | 07 | A2 |
| R3  | 00 | 00 |
| R4  | 0C | BE |
| R5  | 0C | BE |
| R6  | 37 | 0A |
| R7  | 00 | 20 |
| R8  | 00 | 02 |
| R9  | 04 | 14 |
| R10 | 04 | 80 |
| R11 | 3C | 10 |
| R12 | 10 | 00 |
| R13 | 0C | 96 |
| R14 | 02 | 80 |
| R15 | 12 | 01 |

MAINLOOP\_A:

0938 8720

093A DCD8

MAINLOOP\_B:

093C DBC1

093E 5BE1

0940 0771

0942 C703

0944 F009

0946 DCD8

0948 0CC3

094A 5CD1

094C 0661

094E C603

0950 F019

0952 0203

0954 D021

0956 09AC

0958

LBI R7, #32 → R7 = 32

MOVE R12, (R13) → R12 = MEM[R13]

MOVE R11, (R12)+

MOVE (R14)+, R11

DEC R7

SZ R7

BRA MAINLOOP\_B

MOVE R12, (R13)

INC2 R12

MOVE (R13)+, R12

DEC R6

SZ R6

BRA MAINLOOP\_A

CALL SmallDelay, R2

|      |      |
|------|------|
| 0C96 | 1000 |
| 0C98 | 1468 |
| 0C9A | 18D0 |
| 0C9C | 1D38 |
| 0C9E | 21A0 |
| 0CA0 | 2608 |
| 0CA2 | 2A70 |
| 0CA4 | 2ED8 |
| 0CA6 | 3340 |
| 0CA8 | 37A8 |

**R6** for number of rows, **R7** for number of 2-byte groups to copy.

**R12** is the address of the starting row of the banner portion to be copied onto the screen.



|     | HI   | LO |
|-----|------|----|
| R0  | 0958 |    |
| R1  | 09F0 |    |
| R2  | 09F8 |    |
| R3  | 0000 |    |
| R4  | 0CBE |    |
| R5  | 0CBE |    |
| R6  | 3700 |    |
| R7  | 0000 |    |
| R8  | 0002 |    |
| R9  | 0414 |    |
| R10 | 0480 |    |
| R11 | 0000 |    |
| R12 | 0100 |    |
| R13 | 0CAA |    |
| R14 | 0500 |    |
| R15 | 1201 |    |

|             |      |      |                                       |
|-------------|------|------|---------------------------------------|
| MAINLOOP_A: |      |      |                                       |
| 0938        | 8720 | LBI  | R7, #32 → R7 = 32                     |
| 093A        | DCD8 | MOVE | R12, (R13) → R12 = MEM[R13]           |
| MAINLOOP_B: |      |      |                                       |
| 093C        | DBC1 | MOVE | R11, (R12)+ → R11=MEM[R12], R12=R12+2 |
| 093E        | 5BE1 | MOVE | (R14)+, R11 → MEM[R14]=R11, R14=R14+2 |
| 0940        | 0771 | DEC  | R7 → R7 = R7-1                        |
| 0942        | C703 | SZ   | R7 → Skip If (R7 == 0)                |
| 0944        | F009 | BRA  | MAINLOOP_B ; (next column this row)   |
| 0946        | DCD8 | MOVE | R12, (R13) → R12 = MEM[R13]           |
| 0948        | 0CC3 | INC2 | R12 → R12 = R12+2 (shift row)         |
| 094A        | 5CD1 | MOVE | (R13)+, R12 → MEM[R13]=R12, R13=R13+2 |
| 094C        | 0661 | DEC  | R6 → R6 = R6-1                        |
| 094E        | C603 | SZ   | R6 → Skip If (R6 == 0)                |
| 0950        | F019 | BRA  | MAINLOOP_A ; (next row)               |
| 0952        | 0203 | CALL | SmallDelay, R2 ; (uses R12)           |
| 0954        | D021 |      |                                       |
| 0956        | 09AC |      |                                       |
| 0958        | ...  |      |                                       |

|      |      |
|------|------|
| 0C96 | 1002 |
| 0C98 | 146A |
| 0C9A | 18D2 |
| 0C9C | 1D3A |
| 0C9E | 21A2 |
| 0CA0 | 260A |
| 0CA2 | 2A72 |
| 0CA4 | 2EDA |
| 0CA6 | 3342 |
| 0CA8 | 37AA |

Do a MEMCPY for each 64 bytes (32x2) of the pre-computed big\_letter content over to what fits on the screen. After each row, update the pointer so that the next time through, the row will be shifted over by two characters. When all the rows are done, perform a brief delay.

|     | HI   | LO |
|-----|------|----|
| R0  | 0958 |    |
| R1  | 09F0 |    |
| R2  | 09F8 |    |
| R3  | 0000 |    |
| R4  | 0CC0 |    |
| R5  | 0D80 |    |
| R6  | 3700 |    |
| R7  | 0000 |    |
| R8  | 0002 |    |
| R9  | 0414 |    |
| R10 | 047E |    |
| R11 | 0000 |    |
| R12 | 0100 |    |
| R13 | 0CAA |    |
| R14 | 0500 |    |
| R15 | 0000 |    |

```

MAINLOOP_A:
0938 8720 LBI R7, #32 → R7 = 32
093A DCD8 MOVE R12, (R13) → R12 = MEM[R13]
MAINLOOP_B:
093C DBC1 MOVE R11, (R12)+ → R11=MEM[R12], R12=R12+2
093E 5BE1 MOVE (R14)+, R11 → MEM[R14]=R11, R14=R14+2
0940 0771 DEC R7 → R7 = R7-1
0942 C703 SZ R7 → Skip If (R7 == 0)
0944 F009 BRA MAINLOOP_B ; (next column this row)
0946 DCD8 MOVE R12, (R13) → R12 = MEM[R13]
0948 0CC3 INC2 R12 → R12 = R12+2 (shift row)
094A 5CD1 MOVE (R13)+, R12 → MEM[R13]=R12, R13=R13+2
094C 0661 DEC R6 → R6 = R6-1
094E C603 SZ R6 → Skip If (R6 == 0)
0950 F019 BRA MAINLOOP_A ; (next row)
0952 0203 CALL SmallDelay, R2 ; (uses R12)
0954 D021
0956 09AC
0958 ...

```

Several shifts later...

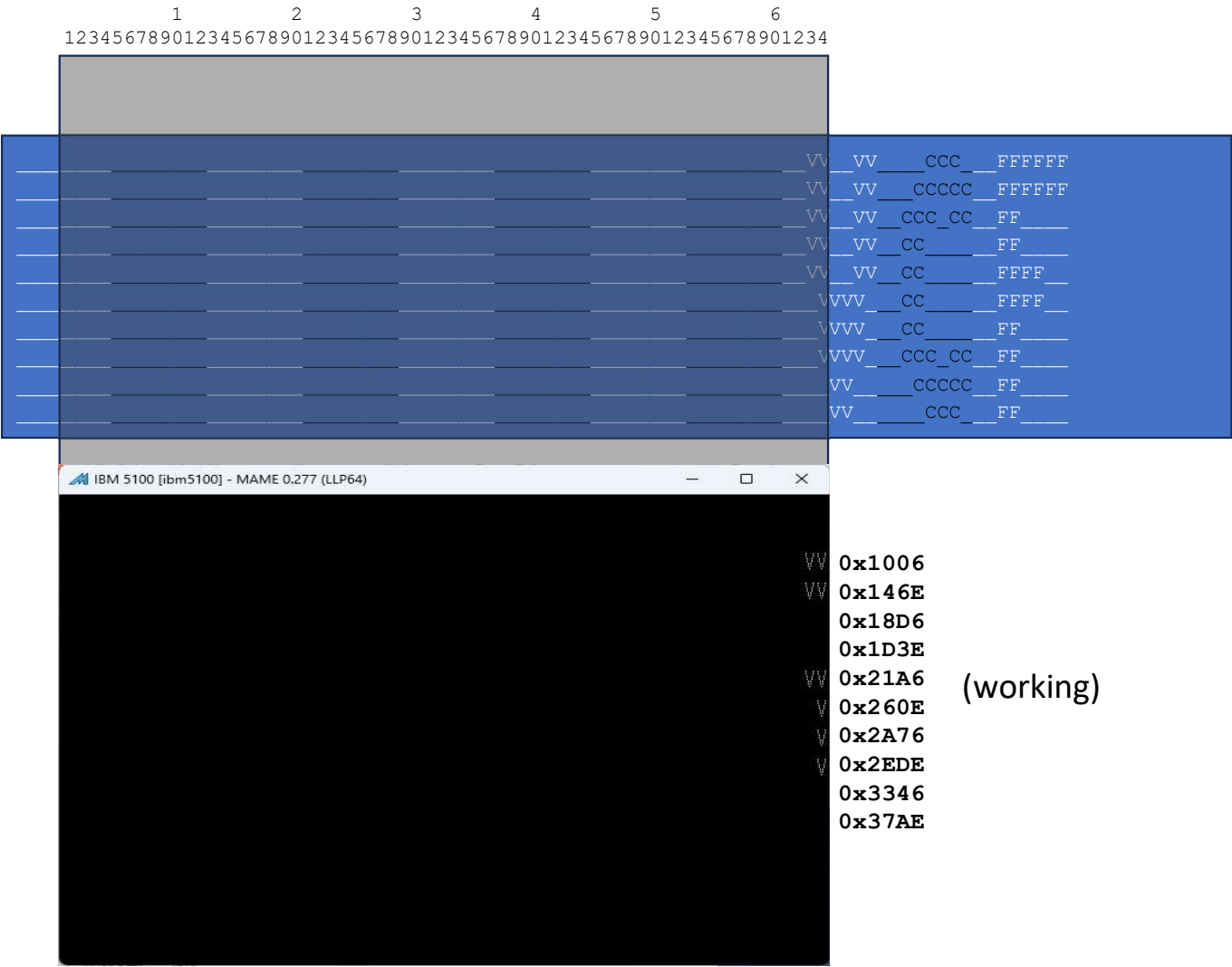
|      |      |
|------|------|
| 0C96 | 1006 |
| 0C98 | 146E |
| 0C9A | 18D6 |
| 0C9C | 1D3E |
| 0C9E | 21A6 |
| 0CA0 | 260E |
| 0CA2 | 2A76 |
| 0CA4 | 2EDE |
| 0CA6 | 3346 |
| 0CA8 | 37AE |

Do a MEMCPY for each 64 bytes (32x2) of the pre-computed big\_letter content over to what fits on the screen. After each row, update the pointer so that the next time through, the row will be shifted over by two characters. When all the rows are done, perform a brief delay.

LINE 1 0x0200  
LINE 2 0x0240  
LINE 3 0x0280  
LINE 4 0x02C0  
LINE 5 0x0300  
LINE 6 0x0340  
LINE 7 0x0380  
LINE 8 0x03C0  
LINE 9 0x0400  
LINE 10 0x0440  
LINE 11 0x0480  
LINE 12 0x04C0  
LINE 13 0x0500  
LINE 14 0x0540  
LINE 15 0x0580  
LINE 16 0x05C0  
0x0600

(initial)

ROW 1 0x1000  
ROW 2 0x1468  
ROW 3 0x18D0  
ROW 4 0x1D38  
ROW 5 0x21A0  
ROW 6 0x2608  
ROW 7 0x2A70  
ROW 8 0x2ED8  
ROW 9 0x3340  
ROW 10 0x37A8



|     | HI   | LO |
|-----|------|----|
| R0  | 096E |    |
| R1  | 096E |    |
| R2  | 0958 |    |
| R3  | 0002 |    |
| R4  | 0000 |    |
| R5  | 0CC0 |    |
| R6  | 3700 |    |
| R7  | 0000 |    |
| R8  | 0002 |    |
| R9  | 0414 |    |
| R10 | 047E |    |
| R11 | 0000 |    |
| R12 | 05CD |    |
| R13 | 0CAA |    |
| R14 | 0500 |    |
| R15 | 0000 |    |

```

0958 DC01 LWI R12, ADDR_SCREEN+(14*64)
095A 0580
095C 8B40 LBI R11, #64 → Lo R11 = 64
095E 0544 MOVE R5, R4 → R5 = R4
PrintStringNextChar:
0960 6F40 MOVB R15, (R4)+ → Lo R15 = MEM[R4], R4=R4+1
0962 CF0C SNS R15 → Skip Not All Set, R15
0964 A007 BRA RESET_STRING
0966 7FC0 MOVB (R12)+, R15 → MEM[R12]=R15, R12=R12+2
0968 0BB1 DEC R11 → R11 = R11-1
096A CB03 SZ R11 → Skip if (R11 = 0)
096C F00D BRA PrintStringNextChar
RESET_STRING:
096E ...

```

**R12** set to the address of the row on the CRT to render the single-row string. This involves copying 64 characters across the entire CRT (**R11**). **R4** is the address into the single row string to copy (incremented during the loop).

|     | HI   | LO |
|-----|------|----|
| R0  | 0990 |    |
| R1  | 097E |    |
| R2  | 0958 |    |
| R3  | 0002 |    |
| R4  | 0CC1 |    |
| R5  | 0D80 |    |
| R6  | 3700 |    |
| R7  | 0000 |    |
| R8  | 0002 |    |
| R9  | 0414 |    |
| R10 | 047D |    |
| R11 | 0000 |    |
| R12 | 05C0 |    |
| R13 | 0CAA |    |
| R14 | 0500 |    |
| R15 | 0000 |    |

RESET\_STRING:

```

096E 0454      MOVE    R4, R5
0970 A400      ADDI    R4, #1
0972 D501 0D80  LWI     R5, TITLE_STRING_TEMP
0976 5458      MOVE    (R5), R4
0978 0AA1      DEC     R10
097A CA03      SZ      R10
097C A011      BRA     NextDelay
097E D401 0CBE  LWI     R4, TITLE_STRING
0982 D501 0D80  LWI     R5, TITLE_STRING_TEMP
0986 5458      MOVE    (R5), R4
0988 D401 0D80  LWI     R4, TITLE_STRING_TEMP
098C D448      MOVE    R4, (R4)
098E 8A80      LBI     R10, #128

```

NextDelay:

```

0990 ...

```

|     | HI | LO |
|-----|----|----|
| R0  | 09 | 9A |
| R1  | 09 | F0 |
| R2  | 09 | 96 |
| R3  | 00 | 03 |
| R4  | 0C | C1 |
| R5  | 0D | 80 |
| R6  | 37 | 00 |
| R7  | 00 | 00 |
| R8  | 00 | 02 |
| R9  | 04 | 14 |
| R10 | 04 | 7D |
| R11 | 00 | 00 |
| R12 | 01 | 00 |
| R13 | 0C | AA |
| R14 | 05 | 00 |
| R15 | 00 | 00 |

NextDelay:

```

0990 0203      CALL    SmallDelay, R2
0992 D021
0994 09AC
0996 A300      ADDI    R3, #1
0998 0F3C      HTL     R15, R3    → Lo R15 = Hi R3
099A CF82      SE      R15, R8    → Skip if Lo R15 == Lo R8
099C F06F      BRA     MAINLOOP    ; not yet reached number of steps to shift
099E C392      SE      R3, R9     → Skip if Lo R3 == Lo R9
09A0 F073      BRA     MAINLOOP    ; perform one scroll cycle
09A2 F095      BRA     MAINLOOP_INIT2 ; Restart big banner from left
09A4 F077      BRA     MAINLOOP    ; not required
09A6 0000      HALT
09A8 0004      NOP
09AA 0004      NOP

```

|     | HI   | LO |
|-----|------|----|
| R0  | PC   |    |
| R1  | ---- |    |
| R2  | ---- |    |
| R3  | ---- |    |
| R4  | ---- |    |
| R5  | ---- |    |
| R6  | ---- |    |
| R7  | ---- |    |
| R8  | ---- |    |
| R9  | ---- |    |
| R10 | ---- |    |
| R11 | ---- |    |
| R12 | ---- |    |
| R13 | ---- |    |
| R14 | ---- |    |
| R15 | ---- |    |

SmallDelay:

```

09AC DC01 01C0 LWI    R12, DELAY_COUNT
09B0 DCC8      MOVE   R12, (R12)

```

delay\_1:

```

09B2 DB01      NOP     ; 1
09B4 01C0      NOP     ; 2
09B6 DF01      NOP     ; 3

```

...

```

09DE 55F8      NOP     ; 23
09E0 0004      NOP     ; 24
09E2 0CC1      DEC     R12
09E4 CC03      SZ      R12
09E6 F031      BRA     delay_1
09E8 0CCC      HTL     R12, R12
09EA 0CC1      DEC     R12
09EC CC0B      SNZ     R12
09EE 0024      RET    R2
09F0 0CCD      LTH     R12, R12
09F2 F03D      BRA     delay_1

```

DELAY\_COUNT = 0x2CFF







## Modifications...

- Adding the Up/Down Key polling to adjust scroll speed

# Ideas for the reserved “Per Character Settings”

- Bits used for “special effects”
  - Alternate “symbol 1” vs “symbol 2” across each banner cycle
  - Counter to store number of full banner cycles (0-255)
  - “Invert” bit (reverse when to draw symbol vs background)
  - Trigger varying speed (to speed up/down delay)